

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

autoAI:recomendador
de vehículos mediante
LLM y web scraping

Curso: 2025/2026

Alumno/a:

Adrián Martínez Granados

Director/es:

Jesús Manuel Almendros Jiménez

ÍNDICE GENERAL

	Página
Resumen y Abstract	XII
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Descripción del problema	1
1.3. Objetivos del trabajo	1
1.3.1. Objetivo general	1
1.3.2. Objetivos específicos	1
1.4. Estructura de la memoria	1
2. Estado del arte	3
2.1. Buscadores y agregadores de coches existentes	3
2.2. Modelos de lenguaje y procesamiento de lenguaje natural	3
2.3. Web scraping para la obtención de datos	3
2.4. Tecnologías web modernas para aplicaciones interactivas	3
2.5. Conclusiones del análisis	3
3. Análisis de requisitos	5
3.1. Descripción general del sistema	5
3.2. Actores del sistema	5
3.3. Requisitos funcionales	5
3.4. Requisitos no funcionales	5
3.5. Casos de uso	5
4. Metodología y planificación	7
4.1. Metodología de desarrollo	7
4.2. Planificación temporal y fases del proyecto	7
4.3. Herramientas y entorno de trabajo	7
4.4. Control de versiones e integración continua	7
5. Diseño del sistema	9

5.1. Arquitectura general	9
5.2. Modelo de datos	11
5.2.1. Diseño de la base de datos	12
5.2.2. Normalización de los datos de los vehículos	14
5.3. Diseño del <i>backend</i> (API REST)	18
5.4. Diseño del pipeline de consulta con IA	19
5.5. Diseño de la interfaz de usuario	24
6. Implementación	31
6.1. Obtención de datos: scraping de km77.com	31
6.1.1. Diseño del <i>spider</i> con Scrapy	31
6.1.2. Control de carga y respeto al portal de origen	33
6.1.3. Automatización periódica del scraping	33
6.2. Proceso ETL y carga en la base de datos	34
6.2.1. Validación y modelos de datos	34
6.2.2. Saneamiento y unificación de vocabularios	34
6.2.3. Persistencia idempotente y tolerante a fallos	36
6.2.4. Esquema de la base de datos	36
6.3. Backend con FastAPI	36
6.3.1. Endpoints del catálogo y comparador	37
6.3.2. El repositorio y la construcción segura de consultas	37
6.3.3. Endpoints de servicio y configuración	37
6.4. Integración de la inteligencia artificial	38
6.4.1. Traducción de lenguaje natural a filtros estructurados	38
6.4.2. Bucle de <i>tool-calling</i> sobre datos reales	39
6.4.3. Generación de respuestas explicadas	40
6.5. Frontend con React	40
6.5.1. Interfaz conversacional	41
6.5.2. Catálogo, filtros y comparador	41
6.5.3. Favoritos e historial de consultas	41
6.5.4. Preferencias de accesibilidad	42
6.6. Contenerización y despliegue con Docker	42
7. Pruebas y evaluación	45
7.1. Estrategia de pruebas	45
7.2. Pruebas del backend y del pipeline de IA	45
7.3. Evaluación de la calidad de los resultados	45
7.4. Discusión de los resultados	45
8. Conclusiones y trabajo futuro	47

8.1. Cumplimiento de los objetivos	47
8.2. Conclusiones	47
8.3. Líneas de trabajo futuro	47
BIBLIOGRAFÍA	47
A. Contenido de una entrada en .bib	51
B. Tipos de referencias	53

ÍNDICE DE FIGURAS

5.1. Diagrama de arquitectura general del sistema.	10
5.2. Esquema físico de la tabla «cars».	14
5.3. Diagrama de flujo del proceso de normalización e ingesta.	17
5.4. Diagrama de secuencia del pipeline de IA. Vista general.	22
5.5. Diagrama de secuencia del pipeline de IA. Bucle de tool-calling.	23
5.6. Diagrama de secuencia del pipeline de IA. Generación del envelope.	23
5.7. Diagrama de secuencia del pipeline de IA. Hidratación de imágenes.	24
5.8. Interfaz conversacional con respuesta de gráfica.	25
5.9. Interfaz conversacional con respuesta de tabla.	25
5.10. Interfaz conversacional con respuesta de imágenes.	26
5.11. Vista de catálogo con filtros.	26
5.12. Vista de favoritos.	27
5.13. Vista de historial.	28
5.14. Vista del panel de preferencias de accesibilidad.	29

ÍNDICE DE TABLAS

5.1. Bloques de atributos de la tabla <code>cars</code>	13
5.2. Vocabulario canónico del atributo <code>combustible</code>	15
5.3. <i>Endpoints</i> de la API REST del <i>backend</i>	18
5.4. Herramientas (<i>tools</i>) disponibles para el modelo de lenguaje.	20
6.1. Ejemplos de saneamiento y unificación de vocabularios: del texto crudo del portal al valor canónico tipado.	35
6.2. Comando de arranque de cada servicio según el destino multietapa (<code>dev/prod</code>) de su <code>Dockerfile</code>	43
A.1. Campos de una entrada en archivo de bibliografía <code>.bib</code>	52
B.1. Tipos posibles de elementos en bibliografía con <code>natbib</code>	54

LISTADOS

6.1. Cascada de <i>callbacks</i> del <i>spider</i> (extracto de <code>km77_spider.py</code>).	32
6.2. Encadenado de <i>crawl</i> e ingesta (<code>run_scrape.sh</code>).	33
6.3. Extracto de <code>_FILTROS_SCHEMA</code> : el enum cerrado del combustible (<code>ai_service.py</code>).	39

ABREVIATURAS

ESI	Escuela Superior de ingeniería
InSo	Ingeniería del Software
SBSE	Search based software engineering
TFG	Trabajo fin de grado
UAL	Universidad de Almería

RESUMEN Y ABSTRACT

En este archivo se incluirá tanto el resumen en castellano como su traducción al inglés. Los dos párrafos estarán ligeramente separados.

El propósito del trabajo en una o dos frases. El diseño y metodología utilizada, los resultados más significativos del trabajo realizado y un breve resumen de las conclusiones. Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG.

Put here the english translation Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG. Los dos párrafos estarán ligeramente separados.

1 INTRODUCCIÓN

1.1 CONTEXTO Y MOTIVACIÓN

1.2 DESCRIPCIÓN DEL PROBLEMA

1.3 OBJETIVOS DEL TRABAJO

1.3.1 Objetivo general

1.3.2 Objetivos específicos

1.4 ESTRUCTURA DE LA MEMORIA

2 ESTADO DEL ARTE

2.1 BUSCADORES Y AGREGADORES DE COCHES EXISTENTES

2.2 MODELOS DE LENGUAJE Y PROCESAMIENTO DE LENGUAJE NATURAL

2.3 WEB SCRAPING PARA LA OBTENCIÓN DE DATOS

2.4 TECNOLOGÍAS WEB MODERNAS PARA APLICACIONES INTERACTIVAS

2.5 CONCLUSIONES DEL ANÁLISIS

3 ANÁLISIS DE REQUISITOS

3.1 DESCRIPCIÓN GENERAL DEL SISTEMA

3.2 ACTORES DEL SISTEMA

3.3 REQUISITOS FUNCIONALES

3.4 REQUISITOS NO FUNCIONALES

3.5 CASOS DE USO

4 METODOLOGÍA Y PLANIFICACIÓN

4.1 METODOLOGÍA DE DESARROLLO

4.2 PLANIFICACIÓN TEMPORAL Y FASES DEL PROYECTO

4.3 HERRAMIENTAS Y ENTORNO DE TRABAJO

4.4 CONTROL DE VERSIONES E INTEGRACIÓN CONTINUA

5 DISEÑO DEL SISTEMA

En este capítulo se describe el diseño de *autoAI* en un nivel intermedio de abstracción: a partir de los requisitos recogidos en el capítulo anterior, se definen la arquitectura global del sistema, la estructura de los datos que lo sustentan, los servicios que lo componen y los flujos de información que los conectan. El objetivo es justificar las decisiones de diseño antes de descender, en el capítulo de implementación, al detalle del código. La exposición recorre las capas del sistema: primero la visión de conjunto y el reparto de responsabilidades entre componentes; después el modelo de datos y su representación en la base de datos; a continuación el diseño del *backend* y de su API REST; seguidamente el pipeline que traduce una consulta en lenguaje natural en una respuesta fundamentada en datos reales; y, por último, el diseño de la interfaz de usuario.

5.1 ARQUITECTURA GENERAL

autoAI se estructura como un sistema distribuido de cuatro **servicios desacoplados**, cada uno con una única responsabilidad y empaquetado en su propio contenedor. Esta separación responde a la heterogeneidad de las tareas del sistema: extraer datos de una fuente web externa, almacenarlos de forma estructurada, exponerlos a través de una API, razonar sobre ellos mediante un modelo de lenguaje y presentarlos en una interfaz interactiva. Cada una de estas actividades tiene un ciclo de vida, unas dependencias tecnológicas y un patrón de ejecución distintos, por lo que aislarlas en componentes separados facilita el mantenimiento, permite escalarlas o sustituirlas por separado y evita que un fallo en una comprometa a las demás.

Los cuatro servicios son los siguientes:

- **Base de datos** (bd): una instancia de *PostgreSQL* que actúa como única fuente de verdad del sistema. Almacena el catálogo de vehículos ya normalizado y es el único componente con estado persistente.
- **Backend** (backend): un servicio construido con *FastAPI* (Python) que centraliza toda la lógica de negocio. Expone la API REST que consume la interfaz, contiene el proceso de normalización y carga de datos, y orquesta el pipeline de inteligencia artificial. Es el único componente que se comunica con la base de datos y con el proveedor externo del modelo de lenguaje.
- **Servicio de scraping** (scraper): un proceso basado en *Scrapy* que extrae periódicamente los datos de los vehículos del portal *km77.com*. No accede directamente a la

base de datos: vuelca el resultado en un archivo y solicita su ingesta al *backend*, que es quien lo valida, normaliza y persiste.

- **Frontend** (*frontend*): una aplicación web de página única (SPA) desarrollada con *React* y servida en producción por un servidor *nginx*. Implementa tanto el asistente conversacional como el catálogo navegable, y consume exclusivamente la API REST del *backend*.

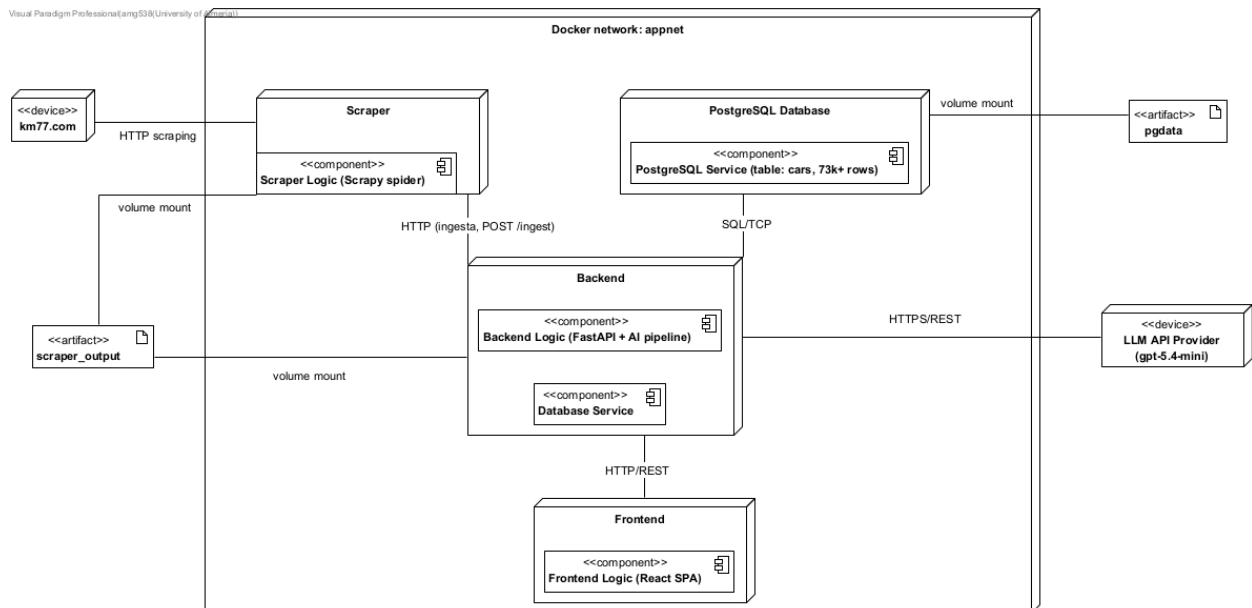


Figura 5.1: Diagrama de arquitectura general del sistema.

La arquitectura se apoya en un **punto de acceso único a los datos**: el *backend* es el único servicio autorizado a leer y escribir en la base de datos. El *frontend* nunca consulta la base de datos directamente, sino que pasa siempre por la API; y el *scraper*, aun produciendo los datos, tampoco los inserta por su cuenta, sino que delega su ingesta en el *backend* mediante una llamada HTTP. Esta decisión concentra en un solo lugar la validación, la normalización y el control de acceso, evita duplicar la lógica de persistencia y garantiza que ningún dato entre en la base de datos sin haber pasado antes por el mismo proceso de saneamiento.

Comunicación entre servicios

Los servicios se comunican mediante dos mecanismos complementarios. El principal es **HTTP/REST**: la interfaz consume los *endpoints* del *backend* para listar el catálogo, comparar vehículos y conversar con el asistente, y el *scraper* invoca el *endpoint* de ingesta cuando termina una extracción. El secundario es un **volumen de disco compartido** entre el *scraper* y el *backend*: el primero escribe en él el archivo con los datos extraídos y el segundo lo lee durante la ingesta. Este reparto evita que el *scraper* necesite conocer el esquema de la base de datos o las credenciales de acceso, y mantiene la frontera entre «producir datos crudos» y «validar y persistir datos» nítidamente trazada.

Todos los servicios residen en una misma red privada virtual y se descubren entre sí por su nombre lógico (bd, backend, etc.), sin necesidad de direcciones fijas. El arranque está ordenado según las dependencias reales: el *backend* no se inicia hasta que la base de datos responde a su comprobación de salud, y ni el *frontend* ni el *scraper* arrancan hasta que el *backend* está operativo. Con ello se evitan los errores transitorios de arranque sin recurrir a esperas arbitrarias. Los detalles de contenerización y orquestación se describen en el capítulo de implementación.

Patrón arquitectónico interno del *backend*

Dentro del *backend*, la organización del código sigue una **arquitectura en capas** que separa la exposición HTTP de la lógica de dominio y del acceso a datos. Se distinguen cuatro niveles:

- La capa de **API** (paquete `api`) define los *endpoints* REST, valida la entrada y traduce las peticiones HTTP en llamadas a la lógica de negocio. No contiene reglas de dominio ni SQL.
- La capa de **servicios** (paquete `services`) concentra la lógica de negocio: la normalización de los datos, el acceso a la base de datos (a través de un *repositorio*) y la integración con el modelo de lenguaje.
- La capa de **modelos** (paquete `models`) define las estructuras de datos con las que trabaja el sistema mediante *Pydantic*, lo que aporta validación automática y una frontera de tipos explícita.
- La capa **núcleo** (paquete `core`) reúne la configuración transversal y la gestión de la conexión a la base de datos.

Esta separación facilita las pruebas (cada capa puede probarse de forma aislada), localiza los cambios (una modificación en el esquema de la base de datos afecta al repositorio, no a los *endpoints*) y mantiene las dependencias dirigidas en un único sentido: la API depende de los servicios y estos de los modelos y el núcleo, nunca a la inversa.

5.2 MODELO DE DATOS

El modelo de datos de *autoAI* tiene un concepto central: la **versión de un vehículo**. El dominio de los automóviles es jerárquico —una *marca* agrupa varios *modelos*, cada modelo se ofrece en distintos *submodelos* o generaciones, y cada submodelo incluye varias *versiones* o acabados con motorizaciones y equipamientos diferentes—, pero es la versión la que constituye la unidad mínima con significado para el usuario: es a ese nivel donde están definidos el precio, la potencia, el consumo y las dimensiones. Por ello, la versión es la entidad sobre la que se construyen las búsquedas, las comparaciones y las recomendaciones.

Esta entidad se representa en el sistema mediante dos estructuras distintas, que corresponden a dos momentos del ciclo de vida del dato:

- Una representación **cruda** (CocheScrap), que recoge los datos tal y como se extraen del portal de origen. Todos sus campos son cadenas de texto, porque así llegan del *scraping*: incluyen unidades («177 CV / 130 kW»), separadores de miles, símbolos de moneda, rangos de fechas y marcadores de dato ausente. Esta representación es fiel a la fuente, pero inservible para consultar o calcular.
- Una representación **normalizada** (Version), que es el modelo canónico del sistema. Cada atributo tiene ya su tipo correcto (numérico, fecha, texto), las unidades se han eliminado, los vocabularios se han unificado y la ausencia de dato se representa de forma explícita. Es la estructura común a todas las capas: la que devuelve la base de datos, la que consume el pipeline de IA y la que recibe la interfaz.

El paso de la primera representación a la segunda es el proceso de normalización, descrito en la sección 5.2.2. Junto a estas dos estructuras existe una tercera familia de modelos, los **bloques de respuesta** del asistente (Block), que no describen vehículos sino el contenido estructurado que el modelo de lenguaje genera (texto, tablas, gráficas e imágenes); se detallan en la sección 5.4.

5.2.1 Diseño de la base de datos

La persistencia se resuelve con una base de datos relacional *PostgreSQL*. El catálogo se almacena en una **única tabla**, *cars*, **desnormalizada de forma deliberada**, en la que cada fila representa una versión completa de un vehículo con todos sus atributos. La jerarquía marca → modelo → submodelo → versión no se modela con tablas separadas y claves foráneas, sino que se aplanan en cuatro columnas de texto dentro de la misma fila (*marca*, *modelo*, *submodelo*, *nombre*).

Esta elección, que se aparta de la tercera forma normal que cabría esperar en un diseño relacional ortodoxo, está justificada por la naturaleza del problema:

- **La carga de trabajo es de solo lectura.** El usuario nunca edita el catálogo; este se regenera por completo desde la fuente. No existe, por tanto, el riesgo de anomalías de actualización que la normalización busca prevenir, que es la principal razón de ser de esta.
- **El patrón de acceso dominante es el filtrado multicriterio sobre una sola entidad.** Casi todas las consultas (tanto del catálogo como de las herramientas de la IA) seleccionan versiones según combinaciones de atributos de la propia versión. Una tabla plana resuelve estas consultas sin ninguna operación de *join*, lo que simplifica el *repositorio* y mejora el rendimiento.
- **Reduce la complejidad.** Para un catálogo de tamaño moderado y que cabe en memoria, un esquema en estrella o totalmente normalizado añadiría complejidad sin un beneficio práctico apreciable.

Estructura de la tabla *cars*

La tabla *cars* reúne veintiséis columnas que se pueden agrupar conceptualmente en cinco bloques, recogidos en la tabla 5.1. La clave primaria es un identificador entero auto-

incremental (*id*), que es el que el sistema usa internamente para referirse a cada vehículo —en las comparaciones, en la recuperación de fichas y en los bloques de imagen generados por la IA—. Además, la columna *url* (la dirección de la ficha del vehículo en el portal de origen) está declarada **UNIQUE** y desempeña el papel de **clave de negocio**: identifica de forma estable una misma versión entre distintas ejecuciones del *scraping*, lo que permite reconocer si un vehículo ya existe y actualizarlo en lugar de duplicarlo.

Bloque		Columnas principales
Identificación		<i>id</i> (PK), <i>marca</i> , <i>modelo</i> , <i>submodelo</i> , <i>nombre</i> , <i>url</i> (única), <i>foto_url</i>
Vigencia		<i>fecha_inicio</i> , <i>fecha_fin</i> (inicio y fin de comercialización)
Precio		<i>precio</i> (en euros)
Carrocería y dimensiones	y	<i>carroceria</i> , <i>puertas</i> , <i>plazas</i> , <i>longitud</i> , <i>anchura</i> , <i>altura</i> , <i>capacidad_maletero</i>
Mecánica y prestaciones	y	<i>combustible</i> , <i>potencia</i> , <i>aceleracion</i> , <i>velocidad_maxima</i> , <i>peso</i> , <i>consumo_medio</i> , <i>traccion</i> , <i>transmision</i> , <i>numero_marchas</i> , <i>capacidad_deposito</i>

Tabla 5.1: Bloques de atributos de la tabla *cars*.

Los tipos de cada columna se han elegido para reflejar la semántica del dato y no su forma de origen. Las magnitudes físicas se almacenan como valores numéricos con la precisión adecuada (por ejemplo, el precio con dos decimales y las dimensiones con un decimal en milímetros), las fechas de comercialización como tipo **DATE**, los pequeños enteros (*puertas*, *plazas*, *número de marchas*) como enteros cortos, y los atributos categóricos (*combustible*, *carrocería*, *tracción*, *transmisión*) como cadenas que contienen siempre un valor del vocabulario canónico definido en la normalización.

Tratamiento de los datos ausentes

El diseño admite **valores nulos** de forma explícita en la mayoría de las columnas de atributos. La fuente de datos no es uniforme: hay vehículos sin maletero (como los *pick-up*), motorizaciones híbridas o eléctricas sin un número discreto de marchas, y campos que el portal no registra. Modelar la ausencia con **NULL** —en lugar de con valores centinela como un cero o una cadena vacía— permite distinguir «este coche pesa cero», que sería un dato erróneo, de «no se conoce el peso de este coche», que es la situación real. Esta distinción se propaga a todo el sistema: el *backend* ordena los resultados colocando siempre los valores nulos al final, y la interfaz los muestra como «dato no disponible» en lugar de inventar una cifra.

Un caso particular es la **vigencia**, modelada con el par de fechas *fecha_inicio* y *fecha_fin*. La convención es que una *fecha_fin* nula significa que la versión *sigue a la venta hoy*; es decir, el valor nulo no representa aquí un dato desconocido, sino un estado del mundo (vehículo aún en comercialización). Esta semántica es la que permite ofrecer al usuario, por defecto, el mercado actual y no versiones descatalogadas, y es un punto que tanto el *backend* como las instrucciones del modelo de lenguaje tratan de forma explícita.

Índices

Para sostener el rendimiento de las consultas más frecuentes sin recurrir a optimizaciones prematuras, el esquema define un conjunto reducido de índices. Se indexan las columnas sobre las que más habitualmente se filtra (marca, modelo, precio, combustible, carrocería, potencia, plazas) y la columna que gobierna el orden por defecto del catálogo (fecha_inicio, ya que las novedades se muestran primero). Se añade además un **índice parcial** sobre las versiones vigentes —las que tienen fecha_fin nula—: al restringir el índice a las filas que cumplen esa condición, se acelera la consulta «solo vehículos a la venta», una de las más recurrentes, y se ocupa mucho menos espacio que con un índice completo.

Visual Paradigm Professional(amg538(University of Almería))

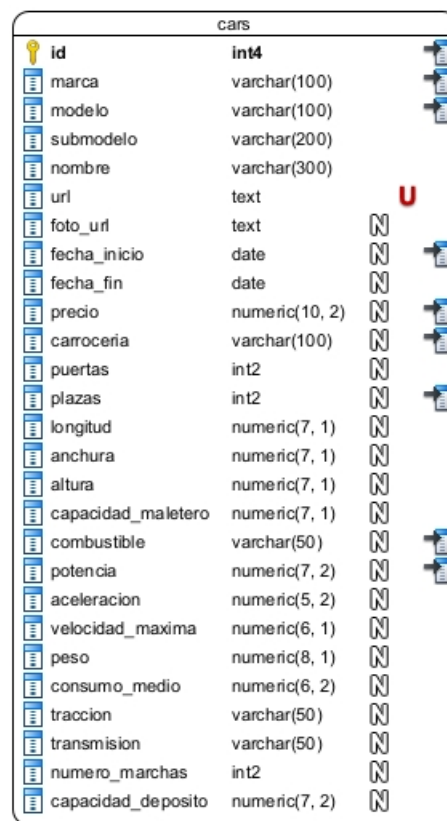


Figura 5.2: Esquema físico de la tabla «cars».

5.2.2 Normalización de los datos de los vehículos

La normalización es una de las partes más delicadas del diseño. Los datos que llegan del *scraping* son fieles a la fuente pero heterogéneos: vienen como texto con unidades incrustadas, con formatos de número propios del español, con rangos de fechas escritos como cadenas y, sobre todo, con vocabularios inconsistentes y datos implícitos. La normalización convierte esos datos en el modelo canónico *Version*, de forma que el resto del sistema —y en particular el modelo de lenguaje— pueda operar sobre datos tipados y con un vocabula-

rio cerrado y predecible. Comprende tres tareas: el saneamiento de los valores escalares, la unificación de los vocabularios categóricos y la persistencia idempotente.

Saneamiento de valores escalares

La parte más mecánica consiste en extraer de cada cadena el valor numérico o la fecha que contiene, descartando todo lo accesorio. Se aplican reglas específicas por tipo de campo: de «177 CV / 130 kW» se conserva únicamente la cifra en caballos; de «4.165 mm» se obtiene el valor en milímetros eliminando el separador de miles; de «7,2 l/100 km» se extrae el número cambiando la coma decimal por punto; y los rangos de fechas del tipo «(03/2021 - 09/2023)» se descomponen en las dos fechas que delimitan la vigencia. Un principio transversal gobierna todo este proceso: ante un valor ausente o un marcador de dato faltante (el portal usa un guion), la conversión **nunca falla ni inventa**, sino que produce un valor nulo. De este modo, una sola versión con un campo problemático no aborta la importación del lote completo.

Unificación de vocabularios categóricos

La dificultad no está en los números, sino en los atributos categóricos, y en particular en el **tipo de combustible**. El portal de origen no clasifica los vehículos por su grado de electrificación: en su campo de combustible solo indica la fuente del motor de combustión (gasolina, gasóleo o combinaciones con gas), y deja ese campo vacío para los eléctricos puros. La información de si un coche es híbrido suave, híbrido autorrecargable o híbrido enchufable no está en ese campo, sino *implícita* en el nombre comercial de la versión.

Para resolverlo se define un **vocabulario canónico cerrado de nueve valores** que codifica, en un único atributo, tanto la fuente fósil como el nivel de electrificación (tabla 5.2). La normalización del combustible determina, por un lado, la base fósil a partir del campo de origen y, por otro, el nivel de electrificación analizando el texto del nombre y el submodelo con expresiones regulares que reconocen los distintos términos comerciales, aplicadas en orden de mayor a menor especificidad (enchufable antes que autorrecargable, y este antes que suave) para evitar clasificaciones erróneas. Este vocabulario es el mismo que después se ofrece al modelo de lenguaje como conjunto cerrado de valores válidos: unificarlo aquí es lo que permite que la IA filtre con precisión más adelante.

Valor canónico	Significado
gasolina	Gasolina pura, sin electrificar
gasoleo	Diésel puro, sin electrificar
gas	<i>Bi-fuel/flex</i> : gasolina con GLP, gas natural o etanol
electrico	100 % eléctrico (BEV)
mhev_gasolina	Microhíbrido (<i>mild hybrid</i> 48 V) de gasolina
mhev_gasoleo	Microhíbrido (<i>mild hybrid</i> 48 V) de diésel
hev_gasolina	Híbrido autorrecargable (<i>full hybrid</i>) de gasolina
phev_gasolina	Híbrido enchufable (PHEV) de gasolina
phev_gasoleo	Híbrido enchufable (PHEV) de diésel

Tabla 5.2: Vocabulario canónico del atributo combustible.

Un tratamiento análogo, aunque más sencillo, recibe el **tipo de carrocería**. El portal emplea sus propias denominaciones (Turismo, SUV/Todoterreno, Turismo familiar, Monovo-

lumen, Descapotable, Pick Up, etc.), que se traducen a un vocabulario canónico de nueve categorías (berlina, suv, compacto, familiar, coupe, cabrio, monovolumen, pickup, furgoneta) mediante una combinación de correspondencias exactas y reglas por subcadena que absorben además sinónimos habituales (*crossover*, *ranchera*, MPV...). El resto de atributos categóricos (tracción y transmisión) se canonizan simplemente pasándolos a minúsculas.

Idempotencia

Las funciones de normalización de vocabulario cumplen la propiedad de **idempotencia**: aplicar la normalización a un valor ya canónico devuelve ese mismo valor sin alterarlo. Como el catálogo se reimporta periódicamente, un dato ya normalizado puede reprocesarse sin corromperse (por ejemplo, sin que un valor ya en forma canónica se transforme de nuevo y deje de ser válido). Gracias a ello, la ingesta es segura de repetir, lo que simplifica la operación del sistema.

Persistencia idempotente y tolerante a fallos

La inserción en la base de datos se ha diseñado también con dos garantías. La primera es la idempotencia a nivel de fila: la sentencia de inserción utiliza la cláusula `ON CONFLICT` sobre la columna `url` de modo que, si una versión ya existe (identificada por su clave de negocio), sus datos se *actualizan* en lugar de provocar un error o un duplicado. Así, el catálogo converge siempre al estado de la última extracción, tanto si se trata de una carga inicial como de una actualización.

La segunda garantía es la **tolerancia a fallos parciales**. Cada versión se inserta dentro de su propio punto de guardado (*savepoint*) anidado, de forma que si una fila concreta provoca un error en la base de datos, solo se revierte esa fila y las demás se conservan. El proceso, además, no se limita a guardar: devuelve un informe detallado del resultado de la importación —cuántos registros se leyeron, cuántos se validaron correctamente, cuántos se normalizaron y cuántos se persistieron, junto con la lista de los que fallaron y por qué—. Este enfoque prioriza la *robustez* sobre la atomicidad estricta: dado que los datos provienen de un *scraping* de una web externa y, por su naturaleza, son imperfectos, es preferible cargar todo lo que sea correcto e informar de lo que no, antes que rechazar un lote completo por unas pocas filas defectuosas.

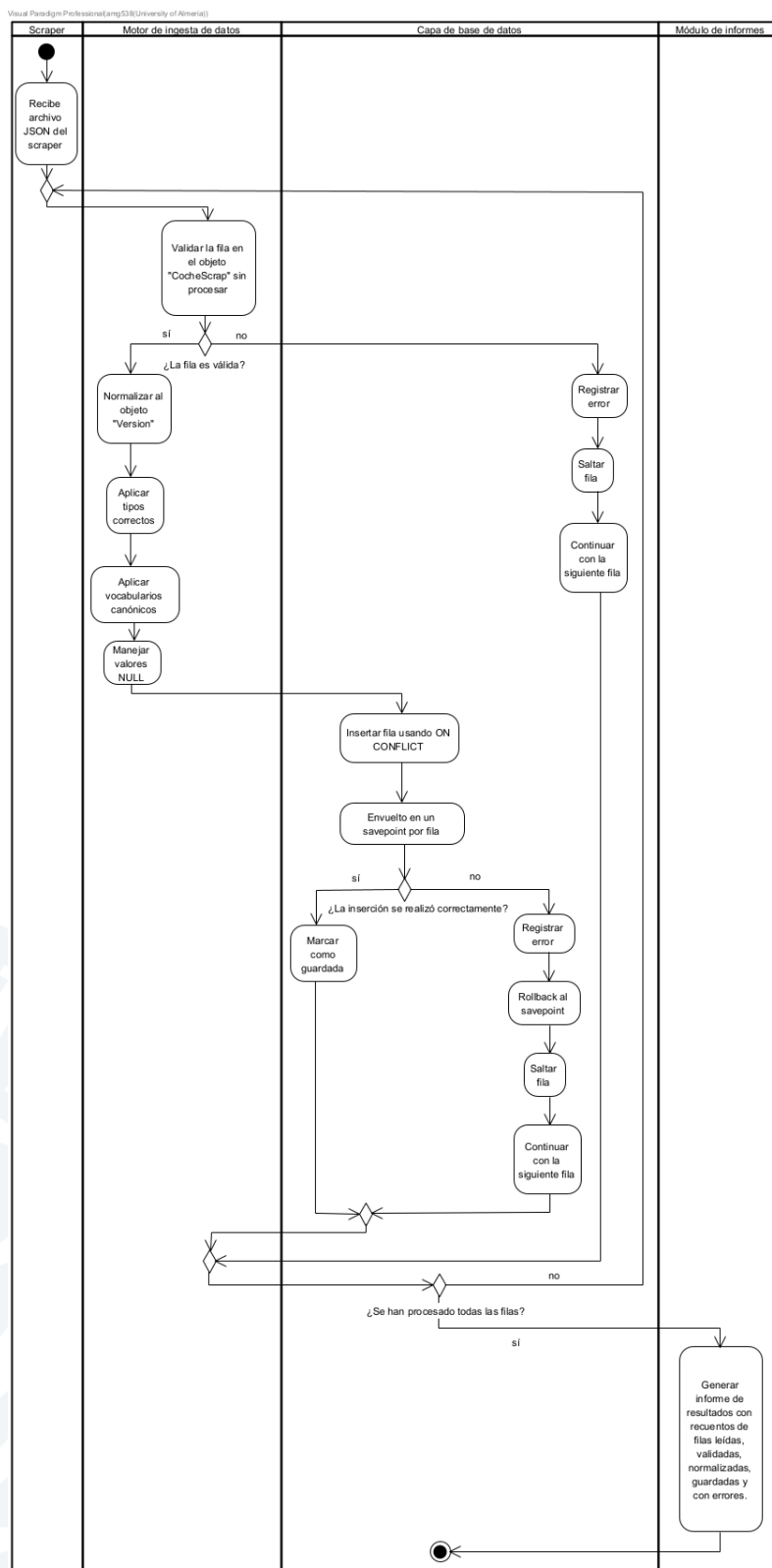


Figura 5.3: Diagrama de flujo del proceso de normalización e ingesta.

5.3 DISEÑO DEL *backend* (API REST)

El *backend* contiene la lógica del sistema y expone sus capacidades a través de una **API REST**. Su diseño persigue tres objetivos: ofrecer a la interfaz un contrato de datos tipado, mantener centralizado y seguro el acceso a la base de datos, y orquestar el pipeline de inteligencia artificial. La API se organiza en dos familias de *endpoints* —los del catálogo y comparador, de naturaleza determinista, y el del asistente conversacional, de naturaleza generativa—, más un par de *endpoints* de servicio. La tabla 5.3 resume el conjunto.

Método	Ruta	Función
GET	/api/cars	Lista de versiones filtrada y paginada (catálogo)
GET	/api/cars/{id}	Ficha completa de una versión
POST	/api/compare	Fichas de varias versiones para comparar
GET	/api/filters/meta	Valores distintos disponibles para los filtros
POST	/chat	Consulta en lenguaje natural al asistente
POST	/ingest	Ingesta de un archivo del <i>scraper</i> (interno)
GET	/health	Comprobación de estado del servicio

Tabla 5.3: *Endpoints* de la API REST del *backend*.

Endpoints del catálogo y comparador

Los *endpoints* del catálogo dan servicio a la navegación clásica de la interfaz. El de listado acepta como parámetros de consulta un conjunto de filtros (marca, modelo, combustible, carrocería, tracción, transmisión y rangos de precio, potencia y plazas) junto con la paginación, y devuelve la lista de versiones que los cumplen. El de ficha individual recupera un vehículo por su identificador, y el de comparación recibe una lista de identificadores y devuelve sus fichas completas. Un *endpoint* auxiliar, el de metadatos de filtros, devuelve los valores realmente presentes en la base de datos para cada atributo categórico, lo que permite a la interfaz poblar dinámicamente sus desplegables sin codificar los valores manualmente.

Las respuestas de estos *endpoints* se ajustan al modelo canónico *Version*. El esquema de la respuesta se declara de forma explícita, con tres consecuencias: la documentación interactiva de la API se genera automáticamente, el *framework* valida la salida, y la interfaz puede definir un tipo «espejo» que refleja esa estructura, de modo que el contrato de datos entre *frontend* y *backend* queda fijado y verificable en ambos extremos.

El repositorio y la construcción segura de consultas

Toda la interacción con la base de datos se concentra en un único módulo, el **repositorio de vehículos**, que actúa como frontera entre la lógica de negocio y el SQL. Este patrón evita que las sentencias SQL se dispersen por el código y permite construir las consultas de forma dinámica a partir de un diccionario de filtros: el repositorio traduce cada filtro presente en una condición de la cláusula *WHERE* y omite los ausentes, soportando tanto igualdades simples

como filtros de pertenencia a un conjunto (por ejemplo, varios tipos de combustible a la vez) y rangos numéricos.

El **tratamiento de la seguridad** de este módulo merece detalle. Todos los valores aportados por el usuario se pasan a la consulta como **parámetros vinculados**, nunca por interpolación de cadenas, lo que elimina por construcción el riesgo de inyección de SQL. En los pocos casos en que es inevitable insertar un identificador en la propia estructura de la consulta —por ejemplo, el nombre de la columna por la que ordenar o agregar—, ese identificador se valida previamente contra una **lista blanca** de valores permitidos: el nombre de ordenación se busca en un diccionario cerrado de órdenes válidos, y los campos y operaciones de agregación se comprueban contra conjuntos predefinidos. Cualquier valor fuera de esas listas se rechaza antes de construir la consulta. De este modo se concilian la flexibilidad de las consultas dinámicas con la imposibilidad de que un valor arbitrario llegue al motor de la base de datos.

Endpoints de servicio y configuración

Completan la API dos *endpoints* de servicio. El de comprobación de estado permite que la orquestación verifique que el *backend* está vivo antes de arrancar los servicios que dependen de él. El de ingesta es el que invoca el *scraper* para solicitar la carga de un archivo; por seguridad, su diseño restringe el acceso al directorio compartido y rechaza cualquier intento de acceder a rutas fuera de él. Toda la configuración sensible del *backend* (la cadena de conexión a la base de datos, la clave del proveedor de IA, los orígenes permitidos para las peticiones de la interfaz) se externaliza mediante variables de entorno y se valida al arranque, de forma que el código no contiene ningún secreto y el comportamiento puede ajustarse entre entornos sin recompilar. Adicionalmente, el *backend* incorpora sendos mecanismos opcionales de limitación de frecuencia de peticiones y de autenticación por clave para los *endpoints* sensibles, desactivados por defecto y activables por configuración.

5.4 DISEÑO DEL PIPELINE DE CONSULTA CON IA

El pipeline de inteligencia artificial es lo que materializa el objetivo principal de *autoAI*: permitir que un usuario exprese sus necesidades en **lenguaje natural** y reciba una recomendación **fundamentada en datos reales**. El problema de diseño consiste en conciliar la flexibilidad de un modelo de lenguaje, que puede interpretar peticiones ambiguas y redactar respuestas naturales, con el **rigor factual** de una base de datos, que es la única fuente legítima de cifras. La solución adoptada es no permitir que el modelo invente datos, sino obligarlo a obtenerlos de la base de datos a través de un conjunto acotado de herramientas, mediante el patrón conocido como *tool-calling*.

De lenguaje natural a filtros estructurados: las herramientas

En lugar de pedir al modelo que genere SQL —lo que sería potente pero arriesgado—, el diseño le ofrece un **catálogo cerrado de cuatro herramientas** de alto nivel que encapsulan las operaciones que el sistema sabe hacer con seguridad (tabla 5.4). La tarea del modelo se reduce entonces a traducir la intención del usuario en una llamada a la herramienta adecuada con los argumentos correctos; es decir, a convertir el lenguaje natural en un conjunto de **filtros estructurados**.

Herramienta	Función
buscar_coches	Busca versiones que cumplen unos filtros, con orden y límite opcionales
obtener_coche	Devuelve la ficha completa de un vehículo por su identificador
comparar	Devuelve las fichas de varios vehículos para compararlos
agregar	Calcula una agregación (recuento, media, mínimo, máximo) sobre un campo, opcionalmente agrupada; es la fuente de las gráficas

Tabla 5.4: Herramientas (*tools*) disponibles para el modelo de lenguaje.

El elemento determinante es la **especificación de los argumentos** de estas herramientas. El esquema de filtros que se entrega al modelo es un contrato estricto: enumera explícitamente los atributos por los que se puede filtrar y, para los categóricos, restringe los valores admisibles al vocabulario canónico definido en la normalización. Aquí se aprovecha la unificación de vocabularios: como el campo de combustible solo admite los nueve valores canónicos, el modelo no puede pedir un «híbrido» genérico ni un «diésel» con otra grafía, sino que debe traducir la intención del usuario a los valores exactos que existen en la base de datos. El propio esquema incluye, a modo de instrucciones, las equivalencias entre el lenguaje coloquial y el vocabulario canónico, lo que guía al modelo en esa traducción.

El bucle de *tool-calling* sobre datos reales

El componente central del pipeline es un **bucle de razonamiento** que alterna entre el modelo de lenguaje y la ejecución de herramientas. Funciona así: se envía al modelo la conversación junto con la descripción de las herramientas; si el modelo decide llamar a una o varias, el *backend* las ejecuta contra la base de datos y le devuelve los resultados; el modelo razona entonces sobre esos datos y decide si necesita más información (refinar la búsqueda, comparar, agregar) o si ya puede responder. El ciclo se repite hasta que el modelo deja de pedir herramientas, con un **límite máximo de iteraciones** que evita bucles infinitos y acota el coste de cada consulta.

Este diseño garantiza la invariante fundamental del sistema: **toda cifra que aparece en una respuesta procede de una consulta real a la base de datos**. El modelo aporta la interpretación, la estrategia de búsqueda y la redacción, pero los datos los pone siempre la base de datos. Sobre esta invariante se articula además un comportamiento experto, codificado en las instrucciones de sistema que acompañan a cada consulta: el modelo recibe directrices sobre cómo interpretar peticiones vagas («algo para la familia», «un coche para ciudad») traduciéndolas a filtros concretos, cómo priorizar por defecto el mercado actual, cómo actuar cuando el usuario filtra por un atributo que el sistema no soporta (explicarlo y ofrecer igualmente una búsqueda aproximada) y cómo tratar casos delicados como las unidades de consumo de los híbridos enchufables.

El diseño distingue entre **consultas objetivas** y **consultas exploratorias**. Cuando el usuario pide algo con un criterio inequívoco («el más barato», «el más potente»), la búsqueda devuelve ese resultado de forma determinista. Cuando la petición es cualitativa y sin un orden objetivo («un coche bonito para ciudad»), el sistema recupera un conjunto de candidatos más amplio y selecciona entre ellos con un muestreo sesgado hacia los más relevantes, de manera

que dos consultas parecidas no devuelvan siempre los mismos vehículos. Así se evita que el asistente resulte repetitivo sin mostrar por ello opciones de peor calidad.

Generación de respuestas estructuradas y explicadas

Una recomendación de coches no se expresa bien solo con texto: a menudo requiere una tabla comparativa, una gráfica de precios o la fotografía del vehículo. Por ello, la respuesta del asistente es un documento estructurado compuesto por una secuencia de **bloques tipados**. Se definen cuatro tipos de bloque: texto (en formato *Markdown*), tabla, gráfica (de barras o radar) e imagen. Una vez ha reunido los datos a través de las herramientas, el modelo emite una respuesta final que se ajusta de forma estricta a este esquema de bloques, lo que permite a la interfaz renderizar cada uno con el componente adecuado.

Este enfoque exige una **segunda fase** en el pipeline, separada del bucle de herramientas: una vez recopilados todos los datos, se realiza una última llamada al modelo solicitándole que produzca el documento de bloques en un formato verificable. El *backend* valida que la respuesta cumple el esquema y, si no lo cumple, reintenta una vez indicándole el error; si tampoco así se obtiene una respuesta válida, se entrega un bloque de texto de reserva en lugar de propagar un fallo a la interfaz.

Dos decisiones de diseño refuerzan en esta fase la integridad de los datos. La primera afecta a las **imágenes**: para evitar que el modelo invente o altere direcciones de fotografías, se le prohíbe emitir URLs; en su lugar, un bloque de imagen solo contiene el identificador del vehículo, y es el *backend* quien, en un paso posterior de «hidratación», recupera de la base de datos la URL real de la foto y la inserta en el bloque (descartándolo si el vehículo no existe o no tiene imagen). La segunda afecta a las **gráficas**: estas se alimentan exclusivamente de la salida de la herramienta de agregación, de modo que las barras de un gráfico reflejan siempre valores calculados por la base de datos y no estimaciones del modelo; las etiquetas, unidades y títulos son metadatos de presentación que nunca modifican las cifras subyacentes. En conjunto, estas medidas trasladan la invariante de «no inventar datos» también a los elementos visuales de la respuesta.

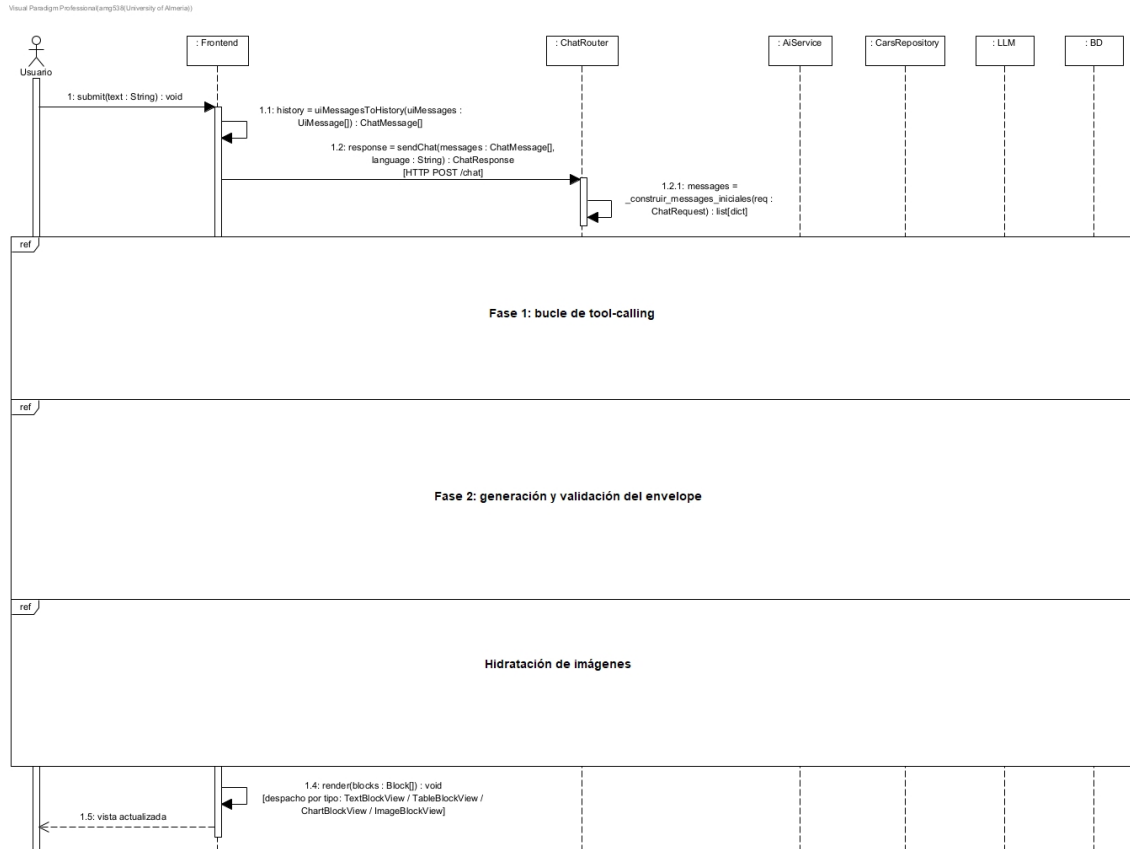


Figura 5.4: Diagrama de secuencia del pipeline de IA. Vista general.

5.4. DISEÑO DEL PIPELINE DE CONSULTA CON IA

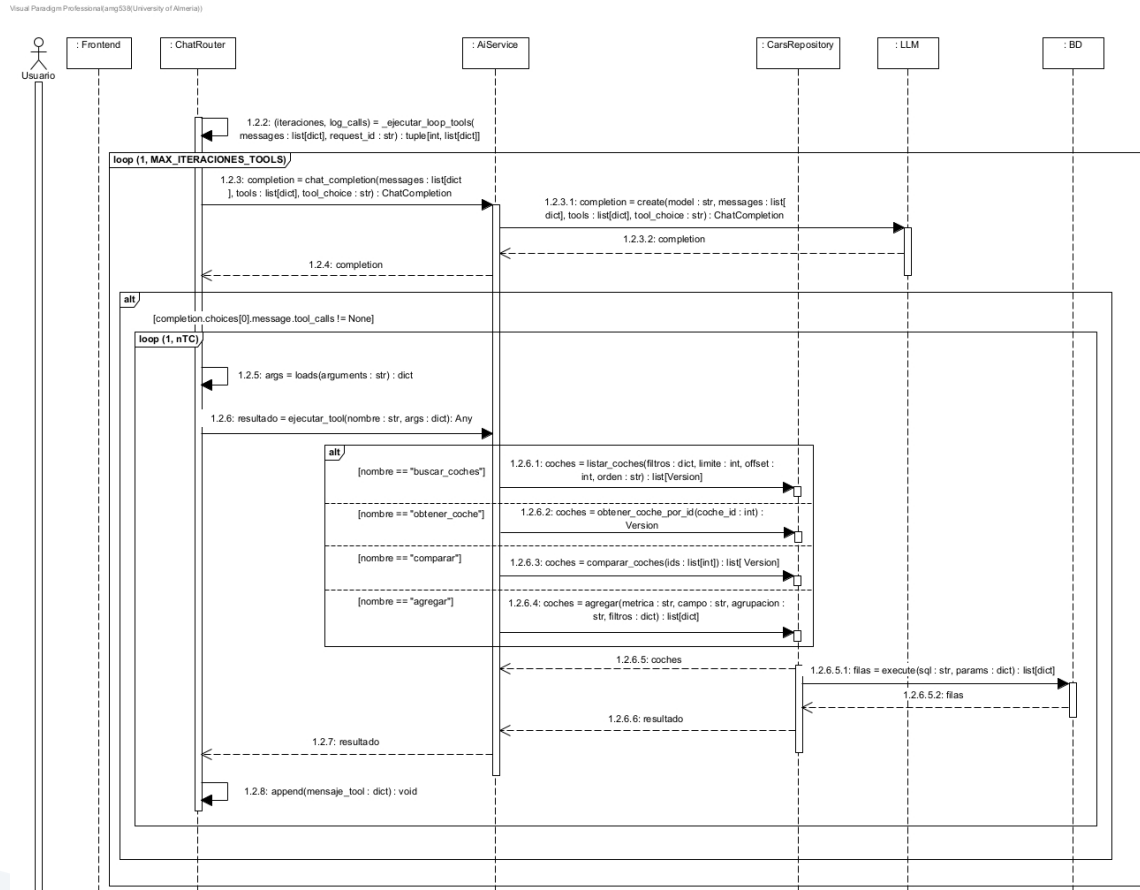


Figura 5.5: Diagrama de secuencia del pipeline de IA. Bucle de tool-calling.

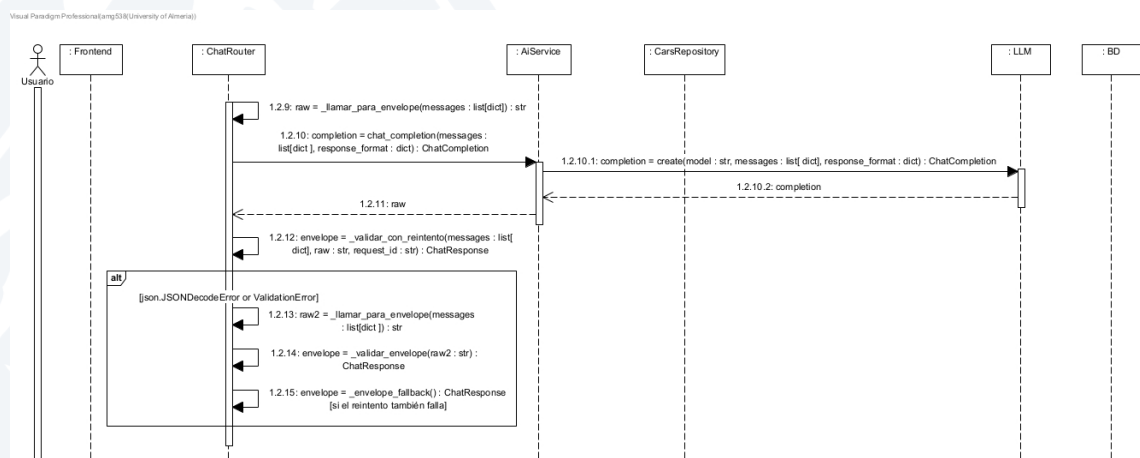


Figura 5.6: Diagrama de secuencia del pipeline de IA. Generación del envelope.

Visual Paradigm Professionals(amg538/University of Almeria)

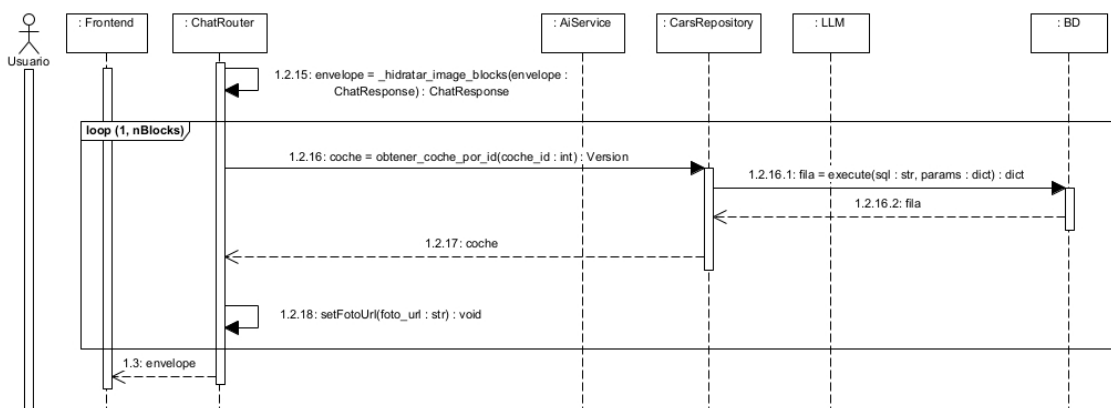


Figura 5.7: Diagrama de secuencia del pipeline de IA. Hidratación de imágenes.

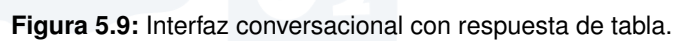
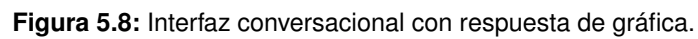
5.5 DISEÑO DE LA INTERFAZ DE USUARIO

La interfaz de usuario se ha diseñado como una **aplicación web de página única**, que actualiza la vista sin recargar la página, organizada en torno a una barra lateral de navegación que da acceso a sus cuatro áreas funcionales: el **asistente conversacional**, el **catálogo**, el **historial** de conversaciones y los **favoritos**. El diseño ofrece dos modos de exploración complementarios sobre el mismo catálogo: uno *conversacional*, guiado por la IA, para el usuario que prefiere expresar sus necesidades con sus propias palabras; y otro *estructurado*, mediante filtros explícitos, para quien prefiere acotar la búsqueda por su cuenta.

Interfaz conversacional

El asistente es la vista principal y se presenta inicialmente con una pantalla de bienvenida que invita a escribir la primera consulta. A partir de ahí, la interacción adopta la forma de una conversación, en la que se alternan los mensajes del usuario y las respuestas del asistente. El problema de diseño propio de esta vista es la **renderización de las respuestas estructuradas**: como la respuesta del *backend* es una secuencia de bloques tipados, la interfaz recorre esos bloques y delega cada uno en un componente especializado —el texto se interpreta como *Markdown*, las tablas se pintan con desplazamiento horizontal, las gráficas se dibujan con una biblioteca de visualización y las imágenes se muestran como fichas con una acción para añadir el vehículo a favoritos—. Este despacho por tipo de bloque permite que una misma respuesta combine, por ejemplo, un párrafo introductorio, una tabla comparativa y una gráfica de precios.

La vista atiende varios detalles de la interacción: mientras el *backend* procesa la consulta se muestra un indicador de escritura, la vista se desplaza automáticamente al último mensaje, los errores de comunicación se presentan como un mensaje del asistente en lugar de como un fallo abrupto, y junto al campo de entrada figura de forma permanente un aviso de que la IA puede cometer errores, en línea con un uso responsable de la herramienta.



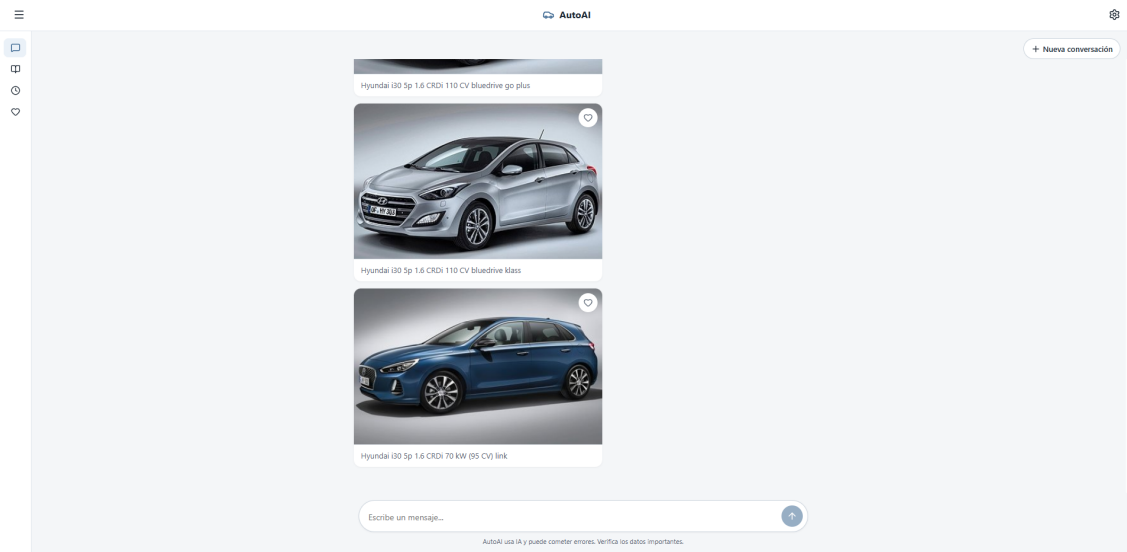


Figura 5.10: Interfaz conversacional con respuesta de imágenes.

Catálogo, filtros y comparador

El catálogo ofrece la vía de exploración estructurada. Presenta los vehículos en una rejilla de tarjetas y, sobre ella, un panel de filtros que se construye dinámicamente a partir de los metadatos que expone el *backend*: los desplegables de marca, combustible, carrocería, tracción y transmisión se pueblan con los valores realmente presentes en la base de datos, lo que evita ofrecer opciones sin resultados. Se complementan con campos para acotar rangos de precio, potencia y número de plazas, y con un buscador de modelo por texto. Los resultados se entregan paginados, y la vista contempla de forma explícita los estados de carga (mostrando una rejilla de marcadores de posición), de error (con opción de reintento) y de búsqueda sin resultados, de modo que la interfaz nunca queda en un estado ambiguo.

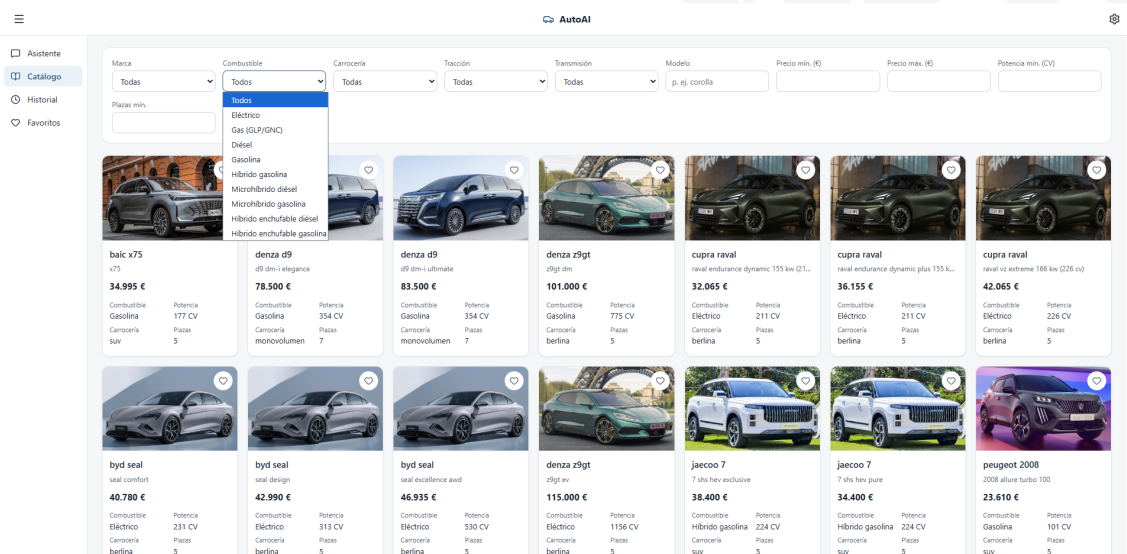


Figura 5.11: Vista de catálogo con filtros.

Favoritos e historial de consultas

Por último, el diseño incluye dos funcionalidades de personalización —**favoritos e historial**—, ambas resueltas en el lado del cliente: se persisten en el almacenamiento local del navegador, sin cuentas de usuario ni almacenamiento en el servidor. Esta opción es coherente con el alcance del proyecto, que no contempla la gestión de usuarios, y simplifica el *backend*, a costa de que la información quede ligada al navegador concreto.

Los favoritos permiten al usuario marcar vehículos de interés (desde las fichas con imagen que genera el asistente o desde el catálogo) y revisarlos después en una vista propia. El historial conserva las conversaciones mantenidas con el asistente, agrupadas en sesiones con un título derivado de la primera pregunta, de manera que el usuario pueda retomarlas o consultarlas más adelante. Para que estas vistas se mantengan siempre sincronizadas con los cambios —incluso entre varias pestañas abiertas simultáneamente—, la lectura del almacenamiento local se integra con el sistema de reactividad de la interfaz, de modo que cualquier modificación se refleja de inmediato en todas las vistas afectadas.

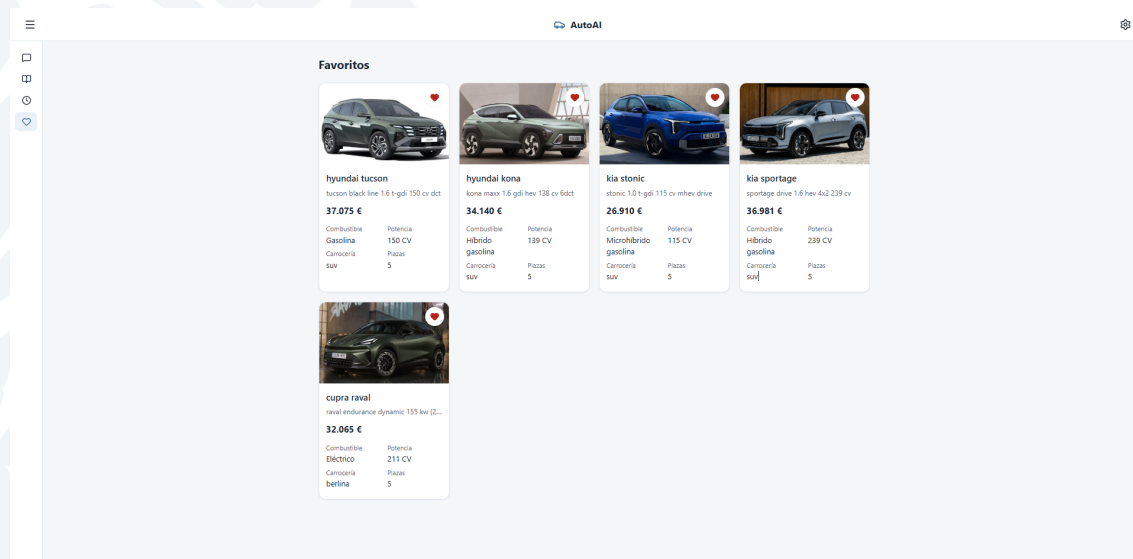


Figura 5.12: Vista de favoritos.

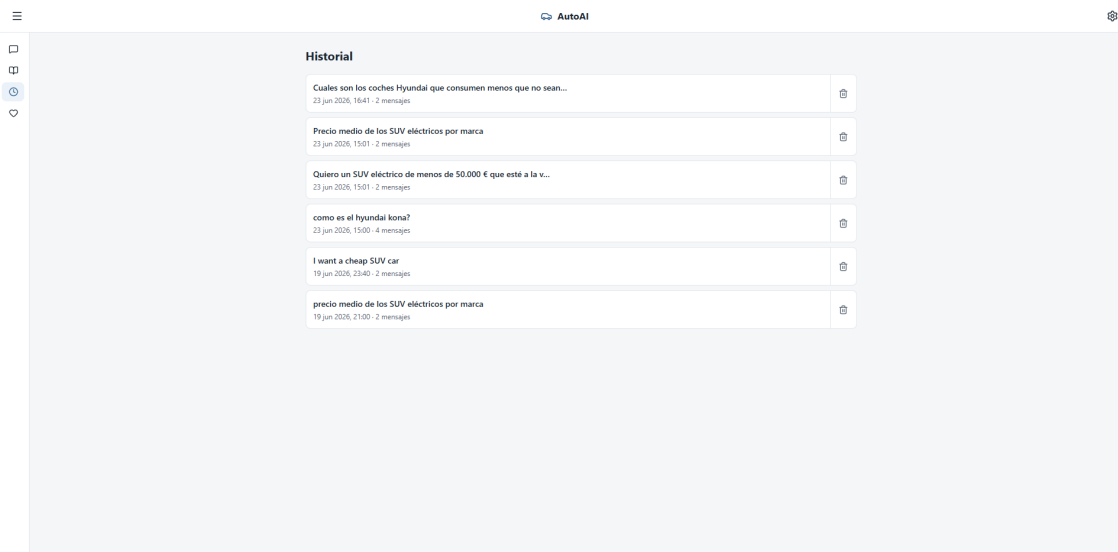


Figura 5.13: Vista de historial.

Preferencias de accesibilidad

La interfaz incorpora un panel de **preferencias de accesibilidad** que permite al usuario adaptar la presentación a sus necesidades. Se presenta como un cajón lateral (*drawer*) accesible —diseñado como diálogo modal con atrapado del foco, cierre con la tecla Esc y navegación completa por teclado— y reúne tres ajustes: el **tema** (claro u oscuro), el **tamaño de la fuente** (reducido, normal o ampliado) y el **idioma** de la interfaz (español o inglés). Coherentemente con la decisión adoptada para favoritos e historial, estas preferencias también se resuelven en el lado del cliente y persisten en el almacenamiento local del navegador, con unos valores por defecto definidos (tema claro, tamaño normal e idioma español) y una lectura defensiva que valida cada campo por separado y recurre al valor por defecto ante cualquier dato corrupto.

El diseño de este panel persigue que cada ajuste tenga un efecto global e inmediato sobre toda la aplicación, para lo cual cada preferencia se traduce en un cambio sobre el documento raíz. El tema alterna un atributo en la raíz del documento que conmuta la paleta de colores de toda la interfaz. El tamaño de fuente, dado que toda la tipografía está expresada en unidades relativas, se controla escalando un único valor en la raíz, lo que redimensiona la interfaz *proporcionalmente* sin romper la maquetación. El idioma, por su parte, ajusta tanto el atributo de idioma del documento como el sistema de internacionalización que traduce los textos de la interfaz; conviene recordar aquí que la localización solo afecta a los textos propios de la aplicación y a la prosa generada por el asistente, nunca a los datos de los vehículos.

Una última consideración de diseño atañe a la **experiencia de arranque**: para evitar el destello visual que se produciría si la aplicación se cargara primero con el tema por defecto y cambiara al tema preferido una vez iniciada, las preferencias guardadas se aplican al documento *antes* de que se monte la interfaz. Así, el usuario percibe desde el primer instante la presentación que había elegido.

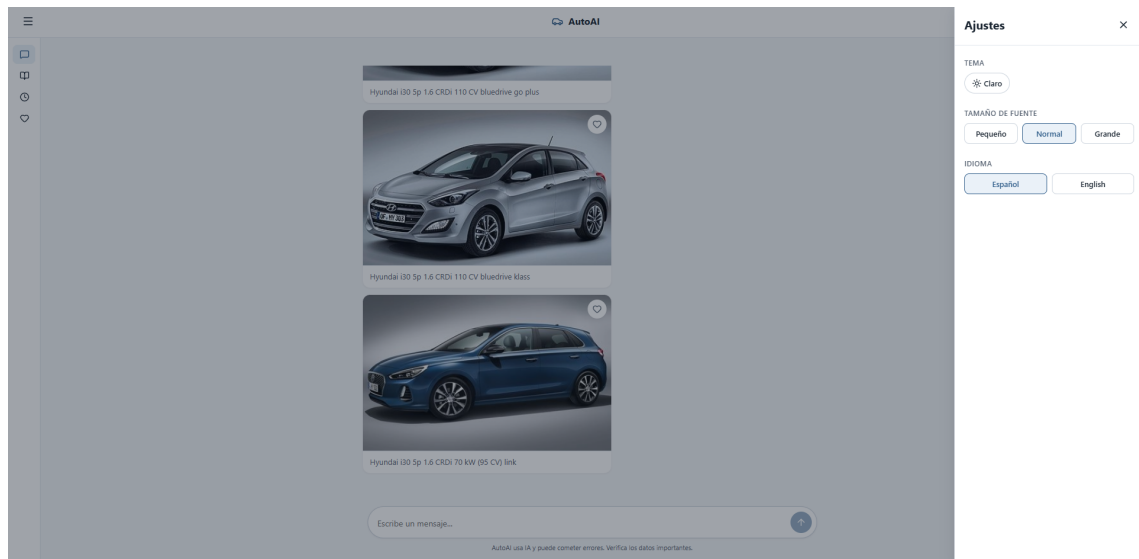


Figura 5.14: Vista del panel de preferencias de accesibilidad.

6 IMPLEMENTACIÓN

Tras el diseño del capítulo anterior, este capítulo expone la implementación: las tecnologías empleadas, las decisiones de codificación relevantes y los fragmentos de código que sustentan cada componente. La estructura del capítulo sigue el recorrido del dato a través del sistema, desde su extracción del portal de origen hasta su presentación en la interfaz, pasando por la base de datos, el *backend*, el pipeline de inteligencia artificial y la contenerización. En cada apartado se justifican las elecciones técnicas y se señalan los puntos del código donde se resuelven los problemas ya anticipados en el diseño.

6.1 OBTENCIÓN DE DATOS: SCRAPING DE KM77.COM

La obtención de datos se ha implementado con *Scrapy*, un *framework* de *scraping* asíncrono para Python. Se eligió por varias razones. Maneja de forma nativa la concurrencia y el encolado de peticiones, e incluye mecanismos de control de carga (*auto-throttle*, retardos entre peticiones) necesarios para no saturar el servidor de origen. Además, permite expresar la navegación por el sitio como una cascada de *callbacks*, que encaja con la estructura jerárquica del portal. Por último, exporta el resultado directamente a JSON, el formato que después consume el *backend*.

6.1.1 Diseño del *spider* con Scrapy

El portal `km77.com` organiza su catálogo de forma jerárquica, y el *spider* (definido en `km77_spider.py`) lo recorre reproduciendo esa jerarquía en cinco niveles encadenados, cada uno con su propio *callback* que extrae enlaces y los sigue hacia el nivel siguiente:

1. **Marcas** (`parse`): a partir de la página índice de coches se extraen los enlaces de cada marca.
2. **Modelos** (`parse_brand`): de la página de cada marca se obtienen sus modelos.
3. **Submodelos** (`parse_model`): de cada modelo se recogen sus generaciones o submodelos. En este nivel se captura la URL de la fotografía (`foto_url`), que solo está disponible aquí.
4. **Versiones** (`parse_submodel`): de cada submodelo se listan sus versiones o acabados, junto con la fecha de comercialización y la capacidad de maletero, que figuran en la tabla del submodelo.

5. **Ficha técnica** (*parse_version*): finalmente, de la página de cada versión se extraen todos los datos técnicos (precio, potencia, dimensiones, consumo, etc.) y se emite el registro final.

La información parcial que se necesita en los niveles inferiores (nombre de la marca, fotografía, fecha y maletero) se traslada entre *callbacks* mediante el diccionario *meta* de las peticiones de Scrapy, de modo que el registro final reúne datos procedentes de varios niveles de la jerarquía.

Las páginas de ficha técnica presentan los datos en tablas cuya estructura no es homogénea: unas etiquetas están en celdas de encabezado (<th>) y otras en celdas de datos (<td>). Para localizar cada dato con independencia de esas variaciones, la extracción combina selectores CSS con expresiones *XPath* que buscan una fila por el texto de su etiqueta. Se definieron dos funciones auxiliares (*xpath_row* y *xpath_cell_all*) que encapsulan ambos casos, lo que hace la extracción robusta frente a la disposición concreta de cada tabla.

Por decisión de diseño, el *spider* no transforma ni valida los datos: extrae cada campo como texto crudo, tal cual aparece en la página (con unidades, símbolos de moneda y marcadores de dato ausente), y lo vuelca sin más procesamiento. El saneamiento se delega íntegramente en el *backend*, lo que mantiene separadas la producción de datos crudos y su validación y persistencia, tal como se justificó en el diseño. El resultado se exporta a *km77_output.json* mediante la configuración de *feeds* de Scrapy.

El listado 6.1 muestra esta cascada: cada *callback* extrae los enlaces de su nivel y los sigue (*response.follow*) hacia el siguiente, trasladando la información parcial en el diccionario *meta*.

```

1 class Km77Spider(scrapy.Spider):
2     name = "km77_spider"
3     start_urls = ["https://www.km77.com/coches"]
4
5     # NIVEL 1 – Marcas
6     def parse(self, response):
7         for brand_link in response.css("a.js-brand-item-link"):
8             brand_name = brand_link.attrib.get("name")
9             yield response.follow(
10                 brand_url,
11                 callback=self.parse_brand,
12                 meta={"brand_name": brand_name},
13             )
14
15     # NIVEL 2 – Modelos por marca
16     def parse_brand(self, response):
17         brand_name = response.meta["brand_name"]
18         for model_tag in response.css("a.d-block"):
19             yield response.follow(
20                 model_url,
21                 callback=self.parse_model,
22                 meta={"brand_name": brand_name, "model_name": model_name},
23             )
24
25     # NIVELES 3 y 4: parse_model -> parse_submodel (analogos)
26     # ...
27
28     # NIVEL 5 – Ficha tecnica de cada version
29     def parse_version(self, response):
30         # xpath_row localiza una celda por el texto de su etiqueta

```

6.1. OBTENCIÓN DE DATOS: SCRAPING DE KM77.COM

```
31 potencia = xpath_row("Potencia maxima")
32 # ... resto de campos como texto crudo, sin normalizar ...
33 yield {
34     "marca": response.meta.get("brand_name"),
35     "potencia": potencia,
36     # ...
37 }
```

Listado 6.1: Cascada de *callbacks* del *spider* (extracto de `km77_spider.py`).

6.1.2 Control de carga y respeto al portal de origen

Como el *scraping* accede a un servicio de terceros, su configuración se ha orientado a no causar perjuicio al portal. El *spider* declara un *user-agent* de navegador realista y un idioma de aceptación coherentes con una visita normal, y aplica varios límites de carga: un retardo entre peticiones (`DOWNLOAD_DELAY`), un tope de peticiones concurrentes y la extensión *auto-throttle* de Scrapy, que ajusta la cadencia según la latencia de respuesta del servidor y la reduce si el portal empieza a tardar más. El proyecto respeta además el fichero `robots.txt` del sitio (`ROBOTSTXT_OBEY`). Con ello se obtienen los datos sin degradar el servicio para otros usuarios.

El *spider* habilita asimismo la persistencia del estado de rastreo (`JOBDIR`), de manera que un *crawl* interrumpido puede reanudarse sin repetir las peticiones ya realizadas, lo que ahorra carga tanto al portal como al propio sistema.

6.1.3 Automatización periódica del scraping

El catálogo de un portal de coches cambia con el tiempo: aparecen modelos nuevos, se descatalogan versiones y se actualizan precios. Para que los datos de *autoAI* no queden obsoletos, el *scraping* se ejecuta de forma periódica y desatendida. La automatización se resuelve dentro del propio contenedor del *scraper* mediante *supercronic*, un planificador tipo *cron* pensado para contenedores (registra su salida por la salida estándar y no requiere un demonio del sistema).

El *script* de arranque del contenedor (`entrypoint.sh`) genera al vuelo una tabla *cron* a partir de la variable de entorno `SCRAPER_CRON`, cuyo valor por defecto es una ejecución semanal, y lanza *supercronic* sobre ella. Así se ajusta la frecuencia del *scraping* por configuración, sin tocar el código ni reconstruir la imagen.

Cada ejecución dispara el *script* `run_scrape.sh`, que encadena las dos fases del proceso: primero lanza el *crawl* de Scrapy y vuelca el resultado en el volumen compartido; después notifica al *backend* mediante una petición HTTP `POST` a su *endpoint* de ingesta con el nombre del archivo recién generado. Conforme al diseño, el *scraper* no inserta nada en la base de datos: solo produce el archivo y delega su ingesta. La llamada usa la opción `-fail-with-body` de `curl`, de modo que un fallo en la ingesta se refleja en el código de salida y queda registrado en los *logs* del contenedor.

El listado 6.2 recoge el *script* `run_scrape.sh`.

```
1 #!/bin/sh
2 set -e
3
```

```
4 OUTPUT="${SCRAPER_OUTPUT:-/data/km77_output.json}"
5 BACKEND="${BACKEND_URL:-http://backend:8000}"
6
7 # /ingest interpreta 'file' relativo a INGEST_DIR (/data): enviamos solo el nombre
8 OUTPUT_NAME="$(basename "${OUTPUT}")"
9
10 # Fase 1: scraping -> volumen compartido
11 cd /app
12 scrapy crawl km77_spider -O "${OUTPUT}"
13
14 # Fase 2: notificar al backend para que ingeste el archivo
15 curl -sS --fail --with-body -X POST -H "Content-Type: application/json" \
16   --data "{\"file\":\"${OUTPUT_NAME}\"}" \
17   "${BACKEND}/ingest"
```

Listado 6.2: Encadenado de *crawl* e ingesta (*run_scrape.sh*).

6.2 PROCESO ETL Y CARGA EN LA BASE DE DATOS

El proceso de extracción, transformación y carga (ETL) reside íntegramente en el *backend*, concentrado en el módulo *services/normalization.py*, y se activa cuando el *scraper* invoca el *endpoint* */ingest*. La función orquestadora, *ingest_desde_archivo*, encadena cuatro etapas (lectura, validación, normalización y persistencia) y devuelve un informe detallado de cada una. Las tres últimas se ejecutan de forma independiente para cada registro, lo que permite aislar los fallos individuales sin abortar el lote completo, conforme a una de las garantías del diseño.

6.2.1 Validación y modelos de datos

El sistema utiliza *Pydantic* para definir y validar todas sus estructuras de datos. El ciclo de vida del dato se modela con dos clases, que corresponden a los dos momentos descritos en el diseño:

- *CocheScrap* (en *models/scraped.py*) es la representación cruda: todos sus campos son de tipo *str*, porque así llegan del *scraping*. Al construir un *CocheScrap* a partir de cada elemento del JSON, *Pydantic* verifica que están todos los campos esperados; un registro malformado lanza una excepción que se captura y se registra como error de validación, sin interrumpir el resto.
- *Version* (en *models/normalized.py*) es el modelo canónico del sistema, con cada atributo ya tipado (*int*, *float*, *datetime.date*, *str*) y con los campos que pueden faltar declarados como opcionales (*| None*). Es la estructura común a todas las capas: la que produce la normalización, la que devuelve la base de datos y la que se serializa hacia la interfaz.

6.2.2 Saneamiento y unificación de vocabularios

La transformación de *CocheScrap* a *Version* la realiza la función *normalizar_coche*, apoyada en funciones de parseo especializadas por tipo de campo. La parte mecánica corresponde a los parsers escalares, que extraen de cada cadena el valor numérico o la fecha

que contiene y descartan lo accesorio: `parse_potencia` conserva la cifra en caballos de «177 CV / 130 kW»; `parse_dimension_mm` elimina el separador de miles de «4.165 mm»; `parse_consumo` retira las unidades de «7,2 l/100 km» y normaliza la coma decimal; y `parse_fechas` descompone un rango «(03/2021 - 09/2023)» en sus dos fechas. Todos ellos se construyen sobre una función común, `parse_opcional`, que aplica el principio transversal del diseño: ante un valor ausente o el marcador de dato faltante del portal (un guion), la conversión no falla ni inventa, sino que devuelve `None`.

Más delicada es la unificación de los vocabularios categóricos, y en particular la del combustible. Como el portal no clasifica los vehículos por su grado de electrificación, sino que deja esa información implícita en el nombre comercial, la función `normalizar_combustible` la deriva en dos pasos: primero determina la base fósil a partir del campo de origen (`_base_fosil`) y después analiza el texto del nombre y el submodelo con tres expresiones regulares (`_RX_PHEV`, `_RX_HEV`, `_RX_MHEV`) que reconocen los términos comerciales. Estas se aplican en orden de mayor a menor especificidad (enchufable antes que autorrecargable, y este antes que microhíbrido) para evitar clasificaciones erróneas, y el resultado es uno de los nueve valores del vocabulario canónico. La normalización de la carrocería (`normalizar_carroceria`) sigue una estrategia análoga, con una tabla de correspondencias exactas y reglas por subcadena que absorben sinónimos.

Estas funciones son idempotentes: al detectar la base fósil reconocen tanto los valores crudos del portal como los ya canónicos, de modo que reprocesar un dato ya normalizado devuelve el mismo valor. Gracias a ello, la reimportación periódica del catálogo es segura de repetir.

La tabla 6.1 recoge varios ejemplos del resultado de este proceso, contrastando el valor crudo extraído del portal con el valor ya normalizado y tipado que se persiste en la base de datos.

Campo	Entrada cruda (texto)	Salida normalizada
potencia	"177 CV / 130 kW"	177.0 (float)
longitud	"4.165 mm"	4165.0 (float)
consumo_medio	"7,2 l/100 km"	7.2 (float)
precio	"35.900 €"	35900.0 (float)
fecha_inicio	"(03/2021 - 09/2023)"	2021-03-01,
fecha_fin		2023-09-01
capacidad_maletero	"_"	NULL
numero_marchas	"Múltiples" (e-CVT)	NULL
combustible	"Gasolina" + nombre con «e-Hybrid»	phev_gasolina
combustible	consumo en "kWh/100 km" (unidad eléctrica)	electrico
carroceria	"SUV/Todoterreno"	suv

Tabla 6.1: Ejemplos de saneamiento y unificación de vocabularios: del texto crudo del portal al valor canónico tipado.

6.2.3 Persistencia idempotente y tolerante a fallos

La carga en la base de datos (`guardar_en_bd`) se implementa con *psycopg* (versión 3) y aplica las dos garantías del diseño. La idempotencia a nivel de fila se consigue con una sentencia `INSERT ... ON CONFLICT (url) DO UPDATE`: si una versión ya existe, identificada por su clave de negocio (la `url`), sus datos se actualizan en lugar de provocar un duplicado, de manera que el catálogo converge al estado de la última extracción. Todos los valores se pasan a la consulta como parámetros con nombre (`%(campo)s`), nunca por interpolación de cadenas.

La tolerancia a fallos parciales se logra con un *savepoint* anidado por cada inserción, mediante el gestor de contexto `conn.transaction()` de *psycopg* dentro del bucle de filas. Si una fila concreta provoca un error en la base de datos, solo se revierte su *savepoint*; las filas correctas previas se conservan y el contador de insertados permanece exacto. El error se acumula en una lista de fallos que forma parte del informe final. Este informe, devuelto por `ingest_desde_archivo` y propagado como respuesta del *endpoint* `/ingest`, recoge cuántos registros se leyeron, validaron, normalizaron y guardaron, junto con la lista de los que fallaron y su motivo. Se prioriza así la robustez sobre la atomicidad estricta: ante datos imperfectos procedentes de una web externa, es preferible cargar todo lo correcto e informar de lo demás que rechazar el lote entero.

6.2.4 Esquema de la base de datos

La base de datos es una instancia de *PostgreSQL* cuyo esquema se define en `bd/init.sql`, un *script* que el contenedor de *PostgreSQL* ejecuta automáticamente en su primera inicialización. Implementa la única tabla `cars`, desnormalizada según se justificó en el diseño, con sus veintiséis columnas, la clave primaria `id`, la restricción `UNIQUE` sobre `url` y el conjunto de índices sobre las columnas de filtrado más habituales. Entre estos destaca el **índice parcial** `idx_cars_vigentes`, restringido a las filas con `fecha_fin IS NULL`, que acelera la consulta «solo vehículos a la venta» ocupando una fracción del espacio de un índice completo. Los índices que dan soporte al orden por defecto y al filtro de vigencia se recogen además en una migración idempotente (`001_indices_fecha.sql`), aplicable sobre una base ya poblada sin recrear el volumen de datos.

El acceso desde el *backend* se gestiona con un **pool de conexiones** (`psycopg_pool`), configurado en `core/db.py`, que se abre al arrancar la aplicación y se reutiliza entre peticiones. Las conexiones se configuran para devolver las filas como diccionarios (`dict_row`), lo que permite construir directamente los modelos *Version* a partir de cada fila.

6.3 BACKEND CON FASTAPI

El *backend* se ha implementado con *FastAPI*, un *framework* web de Python que aprovecha las anotaciones de tipos y se integra de forma nativa con *Pydantic*. Esa integración motivó su elección: al declarar los modelos de entrada y salida con tipos, *FastAPI* valida las peticiones, serializa las respuestas y genera documentación interactiva de la API sin código adicional. El servidor se ejecuta sobre *uvicorn* (servidor ASGI) y, en producción, con varios procesos de trabajo.

La aplicación se compone en `main.py`, que crea la instancia de FastAPI, registra los enrutadores de las familias de *endpoints*, configura el *middleware* de CORS a partir de los orígenes permitidos por configuración y gestiona el ciclo de vida del *pool* de conexiones mediante un *lifespan*, que lo abre al arrancar y lo cierra al terminar. La estructura del código respeta la arquitectura en capas del diseño: la capa de API (paquete `api`), la de servicios (`services`), la de modelos (`models`) y el núcleo transversal (`core`).

6.3.1 Endpoints del catálogo y comparador

Los *endpoints* del catálogo se definen en `api/cars.py`. El de listado (`GET /api/cars`) declara cada filtro como un parámetro de consulta con su tipo y sus restricciones (por ejemplo, los rangos de precio y potencia se validan como no negativos, y la paginación se acota a un máximo por página), recoge los filtros en un diccionario y delega en el repositorio. Se completan con el *endpoint* de ficha individual (`GET /api/cars/{id}`), que responde con un error 404 si el vehículo no existe; el de comparación (`POST /api/compare`), que recibe una lista de identificadores; y el de metadatos de filtros (`GET /api/filters/meta`), que devuelve los valores realmente presentes en la base de datos para poblar dinámicamente los desplegables de la interfaz. Todas estas respuestas se declaran con `response_model`, de modo que FastAPI valida la salida contra el modelo `Version` y fija el contrato de datos con el *frontend*.

6.3.2 El repositorio y la construcción segura de consultas

Toda la interacción con la base de datos se concentra en `services/cars_repository.py`, que actúa como frontera entre la lógica de negocio y el SQL. Las consultas del catálogo se construyen dinámicamente: la función `_construir_where` traduce cada filtro presente en el diccionario en una condición de la cláusula `WHERE` y omite los ausentes. Admite igualdades simples, rangos numéricos y filtros de pertenencia a un conjunto, que generan una condición `= ANY(...)` cuando llegan varios valores (por ejemplo, varios combustibles a la vez).

La seguridad es el aspecto más cuidado de este módulo. Los valores aportados por el usuario se pasan siempre como parámetros vinculados (`%(...)s`), nunca por interpolación de cadenas, lo que elimina por construcción el riesgo de inyección de SQL. En los pocos casos en que es inevitable insertar un identificador en la estructura de la consulta (el campo por el que ordenar, o el campo y la operación de una agregación), ese identificador se valida previamente contra una lista blanca: el orden se resuelve buscando la clave en un diccionario cerrado (`ORDENES_VALIDOS`), y la función `agregar` comprueba la métrica, el campo y la agrupación contra conjuntos predefinidos (`METRICAS_AGREGAR`, `CAMPOS_AGREGABLES`, `AGRUPACIONES_VALIDAS`) y lanza una excepción ante cualquier valor fuera de ellos antes de construir la consulta. Esta invariante de seguridad está documentada en el propio código para que no se relaje al añadir nuevos campos. Por último, todos los órdenes colocan los valores nulos al final (`NULLS LAST`) y añaden un desempate por `id`, lo que garantiza una paginación estable.

6.3.3 Endpoints de servicio y configuración

Completan la API el *endpoint* de comprobación de estado (`GET /health`), que la orquestación usa para verificar que el servicio está vivo, y el de ingesta (`POST /ingest`). Este último,

definido en `api/routes.py`, interpreta el nombre de archivo recibido como relativo al directorio compartido y, antes de abrirlo, comprueba que la ruta resuelta sigue estando dentro de ese directorio, rechazando con un error cualquier intento de salir de él (*path traversal*).

La configuración se centraliza en `core/config.py` mediante *pydantic-settings*: la cadena de conexión a la base de datos, la clave del proveedor de IA, el modelo a utilizar y los orígenes CORS se leen de variables de entorno y se validan al arranque, de forma que el código no contiene ningún secreto. Por último, el *backend* incorpora dos mecanismos de seguridad opcionales para los *endpoints* sensibles (`/chat` e `/ingest`), implementados como dependencias de FastAPI en `api/dependencies.py`: una limitación de frecuencia de peticiones por IP y una autenticación por clave de API. Ambos están **inertes por defecto** y solo se activan al definir las variables de entorno correspondientes; la comparación de la clave se hace en tiempo constante (`secrets.compare_digest`) para no filtrar información por el tiempo de respuesta.

6.4 INTEGRACIÓN DE LA INTELIGENCIA ARTIFICIAL

La integración con el modelo de lenguaje se realiza a través de la API de *chat completions* compatible con OpenAI, empleando su biblioteca cliente oficial para Python. La lógica se reparte en dos módulos: `api/chat.py`, que contiene el *endpoint* `/chat` y orquesta el pipeline completo, y `services/ai_service.py`, que encapsula el cliente del modelo, la definición de las herramientas y su ejecución contra el repositorio. Esta separación mantiene la lógica de IA aislada del resto del *backend* y facilita su prueba.

6.4.1 Traducción de lenguaje natural a filtros estructurados

El enfoque se basa en el patrón de *tool-calling*: en vez de pedir al modelo que genere SQL, se le ofrece un catálogo cerrado de cuatro herramientas de alto nivel (`buscar_coches`, `obtener_coche`, `comparar` y `agregar`), declaradas en la lista `TOOLS` de `ai_service.py`. Cada herramienta se describe con un esquema JSON que enumera sus argumentos, y la tarea del modelo se reduce a traducir la intención del usuario en una llamada a la herramienta adecuada con los argumentos correctos.

La pieza determinante es el esquema de filtros (`_FILTROS_SCHEMA`), que actúa como contrato estricto: para los atributos categóricos, restringe los valores admisibles mediante listas `enum` al vocabulario canónico definido en la normalización. Así, el campo de combustible solo acepta los nueve valores canónicos, lo que impide al modelo pedir un «híbrido» genérico o un «diésel» con otra grafía y le obliga a traducir la intención del usuario a los valores que existen en la base de datos. El propio esquema y las listas de `enum` se construyen a partir de las mismas constantes que valida el repositorio (`ORDENES_VALIDOS`, `CAMPOS_AGREGABLES...`), de modo que la descripción que recibe el modelo y la validación del servidor no pueden divergir.

Esta orientación experta se refuerza con un extenso *prompt* de sistema (`SYSTEM_PROMPT`) que codifica el conocimiento de dominio: las equivalencias entre el lenguaje coloquial y el vocabulario canónico, las reglas para interpretar peticiones vagas («algo para la familia», «un coche para ciudad»), la prioridad por defecto del mercado actual, el tratamiento de casos

delicados como el consumo de los híbridos enchufables y el procedimiento a seguir cuando el usuario filtra por un atributo no soportado. A este *prompt* se concatena una instrucción de idioma que afecta únicamente a la prosa generada, nunca a los datos del vehículo.

El listado 6.3 muestra un extracto del esquema de filtros: el atributo `combustible` se declara como un *array* cuyos elementos están restringidos por una lista *enum* al vocabulario canónico, de modo que el modelo no puede emitir un valor fuera de él.

```

1 _FILTROS_SCHEMA = {
2     "type": "object",
3     "properties": {
4         "combustible": {
5             "type": "array",
6             "items": {
7                 "type": "string",
8                 "enum": [
9                     "gasolina", "gasoleo", "gas", "electrico",
10                    "mhev_gasolina", "mhev_gasoleo", "hev_gasolina",
11                    "phev_gasolina", "phev_gasoleo",
12                ],
13            },
14            "description": (
15                "Fuente fosil + electrificacion. NO existe 'hibrido' ni "
16                "'diesel'. Mapea: 'diesel'->['gasoleo'], 'GLP'->['gas']..."
17            ),
18        },
19        # ... resto de filtros (carroceria, precio_min, solo_vigentes, ...) ...
20    },
21    "additionalProperties": False,
22 }

```

Listado 6.3: Extracto de `_FILTROS_SCHEMA`: el *enum* cerrado del combustible (`ai_service.py`).

6.4.2 Bucle de *tool-calling* sobre datos reales

El bucle de razonamiento se implementa en la función `_ejecutar_loop_tools` de `chat.py`. En cada iteración se envía al modelo la conversación junto con la descripción de las herramientas; si el modelo decide llamar a una o varias, el *backend* las ejecuta (`ai_service.ejecutar_tool`) contra la base de datos, añade los resultados a la conversación como mensajes de rol `tool` y reitera. El ciclo continúa hasta que el modelo deja de pedir herramientas, con un límite máximo de iteraciones (`MAX_ITERACIONES_TOOLS`) que acota el coste y evita bucles infinitos; un umbral de advertencia inferior registra en los *logs* las consultas anormalmente largas para su análisis. La ejecución de cada herramienta se envuelve en un `try/except`, de modo que un fallo concreto no aborta la conversación, sino que se devuelve al modelo como un resultado de error sobre el que puede razonar.

Este diseño garantiza la invariante del sistema: toda cifra que aparece en una respuesta procede de una consulta real a la base de datos. El modelo aporta la interpretación y la redacción, pero los datos los pone siempre la base de datos.

Dentro de `buscar_coche` se distingue entre consultas objetivas y exploratorias. Cuando la petición tiene un criterio inequívoco y el modelo solicita un orden explícito («el más barato», «el más potente»), la búsqueda devuelve ese resultado de forma determinista. Cuando la petición es cualitativa y se pide sin orden, se recupera un conjunto de candidatos más amplio y se selecciona entre ellos con un muestreo sesgado (`_submuestra_sesgada`): se asigna

a cada candidato un peso decreciente según su relevancia y se extraen sin reemplazo, de modo que los más relevantes salen con más probabilidad pero dos consultas parecidas no devuelven siempre los mismos vehículos. Así se evita que el asistente resulte repetitivo sin mostrar por ello opciones de peor calidad. La función contempla además que el modelo a veces aplane los argumentos (`_extraer_filtros`) y reconstruye entonces el diccionario de filtros de forma tolerante.

6.4.3 Generación de respuestas explicadas

Una vez recopilados los datos, una segunda fase del pipeline solicita al modelo la respuesta final como un documento estructurado de bloques tipados. Estos se definen en `models/chat.py` como una unión discriminada de Pydantic (Block) con cuatro variantes (texto, gráfica, tabla e imagen) y se exigen al modelo mediante un *response format* de tipo `json_schema` derivado del propio modelo (`_response_format`), lo que fuerza una salida verificable.

La validación de la respuesta es defensiva (`_validar_con_reintento`): el *backend* comprueba que el JSON cumple el esquema y, si no lo cumple, reintenta una vez devolviendo al modelo el mensaje de error concreto; si tampoco así se obtiene una respuesta válida, entrega un bloque de texto de reserva (`_envelope_fallback`) en lugar de propagar un fallo a la interfaz.

Dos decisiones de implementación trasladan la integridad de los datos a los elementos visuales. La primera es la hidratación de imágenes (`_hidratar_image_blocks`): el modelo tiene prohibido emitir URLs de fotografías y solo puede indicar el identificador del vehículo en un bloque de imagen; es el *backend* quien, en un paso posterior, recupera de la base de datos la URL real de la foto y la inserta, y descarta el bloque si el vehículo no existe o no tiene imagen. Para que esa URL no forme parte del esquema que se ofrece al modelo, el campo correspondiente se marca con `SkipJsonSchema`. La segunda afecta a las gráficas: se alimentan exclusivamente de la salida de la herramienta *agregar*, de modo que sus barras reflejan siempre valores calculados por la base de datos. Las etiquetas, unidades y títulos son metadatos de presentación que nunca modifican las cifras subyacentes, una separación que se documenta tanto en el modelo del *backend* como en su espejo del *frontend*.

6.5 FRONTEND CON REACT

La interfaz se ha implementado como una aplicación de página única con *React* (versión 19) y *TypeScript*, construida con la herramienta *Vite*. El uso de *TypeScript* permite definir un tipo `Car` (en `lib/api.ts`) que es el espejo exacto del modelo *Version* del *backend*, de modo que el contrato de datos queda verificado también en el lado del cliente. La capa de comunicación con la API se concentra en ese mismo módulo, que expone funciones tipadas para cada *endpoint* (`sendChat`, `fetchCars`, `fetchCar`, `fetchFiltersMeta`). La aplicación se organiza en torno a un componente `Layout` con una barra lateral que conmuta entre las cuatro vistas funcionales (asistente, catálogo, historial y favoritos), gobernadas por el estado de `App.tsx`.

6.5.1 Interfaz conversacional

La vista del asistente (`ChatAssistant.tsx`) gestiona el estado de la conversación y la comunicación con el *endpoint* `/chat`. Al enviar un mensaje, reconstruye el historial completo en el formato que espera el *backend*, serializando cada respuesta previa del asistente de nuevo a su JSON de bloques, y atiende los detalles de la interacción: muestra un indicador de escritura mientras se procesa la consulta, desplaza la vista automáticamente al último mensaje y, si la comunicación falla, presenta el error como un mensaje del propio asistente en lugar de como un fallo abrupto. Junto al campo de entrada figura de forma permanente el aviso de que la IA puede cometer errores.

El problema propio de esta vista es la renderización de las respuestas estructuradas, resuelto en `MessageBubble.tsx`. El componente recorre la lista de bloques y delega cada uno, según su tipo, en un sub-componente especializado: el texto se interpreta como *Markdown* (con la biblioteca `react-markdown` y soporte de tablas mediante `remark-gfm`); las tablas se pintan con desplazamiento horizontal; las gráficas se dibujan con la biblioteca *Recharts*; y las imágenes se muestran como fichas con un botón para añadir el vehículo a favoritos. Este despacho por tipo permite que una misma respuesta combine, por ejemplo, un párrafo, una tabla y una gráfica.

El componente de gráficas incorpora varias salvaguardas de presentación: recorta de forma determinista a un máximo de barras, oculta la leyenda cuando hay una sola serie, rota las etiquetas del eje cuando hay muchas categorías y, en las gráficas de tipo radar, normaliza cada eje a una escala común y muestra el valor real en la información emergente. Nada de esto altera los datos de origen, solo su presentación. Los colores se leen de variables CSS, de manera que las gráficas siguen el tema activo de la interfaz. El aspecto final de esta vista, con respuestas que combinan distintos tipos de bloque, se mostró en las figuras 5.8, 5.9 y 5.10 del capítulo anterior.

6.5.2 Catálogo, filtros y comparador

La vista de catálogo (`Catalog.tsx`) implementa la exploración estructurada. Al montarse, recupera del *backend* los metadatos de filtros y con ellos puebla dinámicamente los desplegables de marca, combustible, carrocería, tracción y transmisión, de modo que solo se ofrecen opciones que existen en la base de datos. Los filtros de rango y el buscador de modelo se combinan con la paginación, y la vista distingue de forma explícita los estados de carga, error y búsqueda sin resultados, y muestra en cada caso el componente adecuado (una rejilla de marcadores de posición durante la carga, un mensaje con opción de reintento ante un error, etc.). La separación entre los filtros que el usuario edita y los efectivamente aplicados evita relanzar la consulta en cada pulsación: la búsqueda solo se dispara al confirmar. El aspecto de esta vista se mostró en la figura 5.11 del capítulo anterior.

6.5.3 Favoritos e historial de consultas

Las funcionalidades de personalización (favoritos e historial) se resuelven íntegramente en el lado del cliente y persisten en el `localStorage` del navegador, sin cuentas de usuario ni almacenamiento en el servidor, en línea con el alcance del proyecto. Toda esta lógica se centraliza en `lib/storage.ts`.

El reto está en mantener todas las vistas sincronizadas con los cambios, incluso entre varias pestañas abiertas. Se resuelve integrando la lectura del almacenamiento con el sistema de reactividad de React a través del hook `useSyncExternalStore`: cualquier modificación de los favoritos invalida una caché interna y notifica a los componentes suscritos, que se vuelven a renderizar de inmediato; y un *listener* del evento `storage` del navegador propaga además los cambios realizados en otras pestañas. El historial agrupa las conversaciones en sesiones con un título derivado de la primera pregunta del usuario. Tanto la lectura de favoritos como la de sesiones es defensiva: se validan los datos leídos y se descartan las entradas con forma inválida, de modo que un `localStorage` corrupto no rompe la aplicación.

6.5.4 Preferencias de accesibilidad

El panel de preferencias (`SettingsDrawer.tsx`) se implementa como un cajón lateral accesible: declarado como diálogo modal (`role="dialog", aria-modal`), atrapa el foco dentro del panel ciclando con la tecla de tabulación, se cierra con la tecla de escape y devuelve el foco al control de origen al cerrarse. Reúne tres ajustes (tema, tamaño de fuente e idioma) gestionados por un contexto de React (`PreferencesContext.tsx`) que los persiste en `localStorage` con una lectura defensiva campo a campo.

Cada preferencia se traduce en un cambio sobre el documento raíz mediante funciones puras (`applyPreferences.ts`), reutilizadas tanto por el contexto como por el *script* de arranque: el tema conmuta un atributo en la raíz del documento que cambia la paleta de colores; el tamaño de fuente escala un único valor en la raíz, lo que, al estar toda la tipografía expresada en unidades relativas, redimensiona la interfaz proporcionalmente; y el idioma ajusta el atributo `lang` del documento y el sistema de internacionalización (*i18next*), que traduce los textos propios de la aplicación sin tocar nunca los datos de los vehículos. Queda una consideración sobre el arranque: en `main.tsx`, las preferencias guardadas se aplican al documento *antes* de montar React, lo que evita el destello visual que se produciría si la aplicación cargara primero con el tema por defecto y cambiara después. El panel de preferencias se mostró en la figura 5.14 del capítulo anterior.

6.6 CONTENERIZACIÓN Y DESPLIEGUE CON DOCKER

Todo el sistema se empaqueta y se orquesta con *Docker* y *Docker Compose*. Cada uno de los cuatro servicios (base de datos, *backend*, *frontend* y *scraper*) se ejecuta en su propio contenedor, definido por su respectivo `Dockerfile`, y todos comparten una misma red privada virtual (`appnet`) en la que se descubren por su nombre lógico.

Los `Dockerfile` del *backend*, el *frontend* y el *scraper* emplean construcciones multietapa con destinos diferenciados de desarrollo (`dev`) y producción (`prod`). Así se optimiza cada caso: en desarrollo, el *backend* arranca con recarga automática y el *frontend* con el servidor de desarrollo de Vite; en producción, el *backend* se ejecuta con varios procesos de trabajo y el *frontend* se compila a ficheros estáticos que sirve un servidor *nginx* ligero. Las imágenes parten de variantes reducidas (*slim*, *alpine*) y el *backend* ejecuta sus procesos con un usuario sin privilegios.

La orquestación se reparte en tres ficheros de Compose que se combinan: uno base (`docker-compose.yml`) con la definición común de los servicios, los volúmenes y la red; uno de *override* (`docker-compose.override.yml`), que se aplica automáticamente en desarrollo, publica los puertos y monta el código fuente como volumen para la recarga en caliente; y uno de producción (`docker-compose.prod.yml`), que selecciona los destinos `prod` y configura la política de reinicio. Esta separación permite levantar el sistema en uno u otro modo sin duplicar la configuración común.

El fichero base recoge además dos decisiones de orquestación del diseño. La primera es la persistencia: se definen dos volúmenes nombrados, uno para los datos de PostgreSQL (`pgdata`) y otro compartido entre el *scraper* y el *backend* (`scraper_output`), a través del cual el primero entrega el archivo de datos al segundo. La segunda es el arranque ordenado mediante *healthchecks* y dependencias condicionales: el *backend* no se inicia hasta que la base de datos supera su comprobación de salud (`pg_isready`), y ni el *frontend* ni el *scraper* arrancan hasta que el *backend* responde correctamente a su *endpoint* `/health`. Con ello se evitan los errores transitorios de arranque sin recurrir a esperas arbitrarias. Toda la configuración sensible (credenciales de la base de datos, clave del proveedor de IA, frecuencia del *scrapping*, orígenes CORS) se inyecta como variables de entorno, de modo que los secretos quedan fuera de las imágenes y del código.

La diferencia entre los modos de desarrollo y producción se concreta en el comando con el que arranca cada servicio, determinado por el destino multietapa seleccionado. La tabla 6.2 resume ambos casos.

Servicio	Desarrollo (dev)	Producción (prod)
bd	Imagen <code>postgres:16-alpine</code> (sin etapa multietapa)	
backend	<code>uvicorn -reload</code> (recarga en caliente)	<code>uvicorn -workers 4</code>
frontend	<code>vite dev</code> (servidor de desarrollo)	<code>nginx</code> (sirve estáticos compilados)
scraper	<code>sleep infinity</code> (inerte; <i>crawl</i> manual)	<code>supercronic</code> (cron que ejecuta <code>run_scrape.sh</code>)

Tabla 6.2: Comando de arranque de cada servicio según el destino multietapa (`dev/prod`) de su Dockerfile.

7 PRUEBAS Y EVALUACIÓN

7.1 ESTRATEGIA DE PRUEBAS

7.2 PRUEBAS DEL BACKEND Y DEL PIPELINE DE IA

7.3 EVALUACIÓN DE LA CALIDAD DE LOS RESULTADOS

7.4 DISCUSIÓN DE LOS RESULTADOS

8 CONCLUSIONES Y TRABAJO FUTURO

8.1 CUMPLIMIENTO DE LOS OBJETIVOS

8.2 CONCLUSIONES

8.3 LÍNEAS DE TRABAJO FUTURO

BIBLIOGRAFÍA

A CONTENIDO DE UNA ENTRADA EN .BIB

Nombre campo	Descripción
address	Usualmente la dirección de la editorial
author	Nombre(s) del (de los) autor(es)
title	Título del libro
chapter	El número de un capítulo (o sección, etc)
edition	La edición de un libro, por ejemplo,segunda
editor	Nombre(s) del (de los) editor(es)
howpublished	Forma en que fue publicada la obra
institution	Institución responsable de un informe técnico
journal	Nombre del periódico o revista
key	Empleado para la alfabetización, referencias cruzadas y para crear una clave cuando la información del autor no está disponible. No debe confundirse con la etiqueta usada en el cite y que debe colocarse al inicio de la entrada
month	El mes de publicación o, para un trabajo inédito, en el que fue escrito
note	Cualquier información adicional que pueda ayudar al lector
number	El número del periódico, la revista, el informe técnico o del trabajo en una serie
organization	La organización responsable de una conferencia o que publica un manual
pages	Números de páginas
publisher	El nombre de la editorial. No debe confundirse con el editor
school	Nombre de la escuela donde fue escrita una tesis
series	El nombre de una serie o conjunto de libros
title	El título del trabajo
type	El tipo de un informe técnico
volume	El volumen de un periódico o una revista, o de algún libro que conste de volúmenes
year	El año de publicación. Para un trabajo inédito, el año en que fue escrito. Generalmente debe consistir de cuatro dígitos, por ejemplo 200

Tabla A.1: Campos de una entrada en archivo de bibliografía .bib

B TIPOS DE REFERENCIAS

article	Un artículo de un periódico o revista
book	Un libro con una editorial que se indica en forma explícita. Los campos requeridos en este caso son author (autor), editor, title (título), publisher (editorial) y year (año)
booklet	Una obra que está impresa y encuadernada, pero sin una editorial o institución patrocinadora
conference	Lo mismo que inproceedings, incluido para compatibilidad con el lenguaje de marcación Scribe
inbook	Una parte de un libro, que puede ser un capítulo (o sección) o un rango de páginas
incollection	Una parte de un libro que tiene su propio título
inproceedings	Un artículo en las actas de sesiones (proceedings) de una conferencia
manual	Documentación técnica
mastersthesis	Una tesis de maestría (Master thesis) o proyecto fin de carrera
misc	Para uso cuando los demás tipos no corresponden
phdthesis	Una tesis de doctorado (Ph D thesis)
proceedings	Las actas de sesiones de una conferencia
techreport	Un reporte publicado por una escuela u otra institución, usualmente numerado dentro de una serie
unpublished	Un documento que tiene un autor y título, pero que no fue formalmente publicado

Tabla B.1: Tipos posibles de elementos en bibliografía con natbib

ARTEFACTOS SOFTWARE

En este archivo se incluirá tanto el resumen en castellano como su traducción al inglés. Los dos párrafos estarán ligeramente separados.

El propósito del trabajo en una o dos frases. El diseño y metodología utilizada, los resultados más significativos del trabajo realizado y un breve resumen de las conclusiones. Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG.

Put here the english translation Debe ser conciso y presentar los resultados obtenidos tras la ejecución del TFG. Los dos párrafos estarán ligeramente separados.

